

FINANCIAL ECONOMETRICS

MODULE 6 | LESSON 1

UNIT ROOT TESTS

Reading Time 60 minutes

Prior Knowledge Time Series Analysis

Keywords Time-stationarity, Difference-stationarity, Unit Root, Dickey Fuller test, Augmented Dickey Fuller test, Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test

In the last two modules, we learned that stationarity is a key property for time series. For example, ARMA models require the time series to be stationary before modeling. We provided conditions to ensure a time series is stationary in previous lessons. In this lesson, we will introduce some formal tests to check if a time series is stationary. We will first briefly review the definitions of strict and weak stationarities. Then, we will introduce three types of stationarity tests: Dickey-Fuller test, augmented Dickey-Fuller test, and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test. We will finish the lesson with some examples applying these tests.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from arch.unitroot import ADF, KPSS
from statsmodels.tsa.stattools import adfuller
from statsmodels.regression.linear_model import OLS
from scipy import stats
import warnings

plt.rcParams["figure.figsize"] = (16, 9) # Figure size and width
```

```
In [2]: # Download datasets
m6_data = pd.read_csv("M6. goog_eur_10.csv")

# Convert date variable to date format and set index
m6_data["Date2"] = pd.to_datetime(m6_data["Date"], format="%m/%d/%Y")
goog = m6_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
```

1. Weak and Strict Stationarity

In Lesson 1 of Module 4, we introduced the notions of weak stationarity and strict stationarity for time series. If a time series is weak stationary, the mean is constant and independent of time location; the autocorrelation of any two data points in the time series will only depend on the time difference but not on the time locations of the data points. For strict stationarity, the joint probability of the time series data points is the same when time shifts.

The property of stationarity of a time series shows up in many time series models. In an ARMA model, the time series is required to be stationary before modeling. There are also coefficient restrictions in both the Box-Jenkins method and GARCH model to ensure the modeled process is stationary. We have seen the importance of being stationary for a time series.

We also learned from Module 4 that there are two methods to possibly turn a non-stationary time series into a stationary one: detrend the time series or difference the time series. If a non-stationary time series fluctuates along a straight trend in the plot, it means this time series tends to reverse back to its long-term trend after deviating from it. If we can remove this long-term trend and the detrended time series is stationary, then we call the original time series a **trend-stationary time series**.

If we can take a first difference of a non-stationary time series and turn it into a stationary time series, we call this original time series a **difference-stationary time series**. Identifying what type of non-stationary time series we have for analysis will help us to properly prepare the data before modeling.

In Module 4, we usually use ACF and PACF plots or time plots to check if a time series is stationary or non-stationary. Even though these visual methods are very straightforward to understand, oftentimes, these methods are very precise. In the following sections, we will introduce some formal methods to test a time series for stationarity. These methods can provide us with more rigorous checks and results for the stationary character of a time series.

2. Unit Root Tests

A **unit root test** is the most popular testing method to test for stationarity for a time series. There are several methods that belong to unit root tests. In this section, we will introduce three unit root testing methods. First, let's talk about what a unit root is.

2.1 What are Unit Roots?

If a time series has a unit root, the time series is not stationary. But does it have a unit root? Assume x_t follows an AR(1) process:

$$x_t = \alpha_0 + \alpha_1 x_{t-1} + v_t$$

Where v_t is a stationary ARMA process with mean = 0

If $|\alpha_1| > 1$, then x_t is an explosive process. If x_t follows an explosive process, it means that as time goes by, x_t would either keep growing larger and larger or reducing more and more without bounds. Explosive processes are not seen in economics and finance, so we will exclude this scenario in this lesson.

If $|\alpha_1|=1$, then x_t has unit root. We can rewrite x_t as follows by iteration:

$$x_t = \alpha_0 t + x_0 + (v_1 + v_2 + \dots + v_t)$$

It is easy to verify that the mean and variance of x_t will depend on t if x_t has unit root. Hence, x_t is non-stationary.

Even though x_t with a unit root is non-stationary, we can take the first difference of x_t and the difference will be stationary. This conclusion can easily be demonstrated from the following difference operation:

$$x_t - x_{t-1} = \nabla x_t = v_t$$

Let's go through some terminology. A non-stationary time series that can be differenced and become a stationary one is called **integrated non-stationary process**. Specifically, a time series with one unit root is integrated with order one, or $I(1)$. In our case, x_t is $I(1)$. A stationary time series with no unit root is integrated with order zero, or $I(0)$. In our case, v_t is $I(0)$. If we take first difference of $I(1)$, the new time series will be $I(0)$. A non-stationary $I(d)$ time series can be differenced d times and become stationary $I(0)$. We can summarize what we just described as follows:

If $y_t \sim I(0)$, then y_t is a stationary time series.

If $y_t \sim I(1)$, then $y_t - y_{t-1} = (1-B)y_t$ is a stationary time series.

If $y_t \sim I(2)$, then $(1-B)^2 y_t$ is a stationary time series

Random walk is a special case of unit root time series where $\alpha_0 = 0$ and $\alpha_1 = 1$. Random walk with a drift is also a special case of unit root time series where $\alpha_0 \neq 0$ and $\alpha_1 = 1$.

2.2 Dickey Fuller Test

The first unit root test we'll discuss is the Dickey Fuller test. The Dickey Fuller test determines if unit roots exist in a time series. The Dickey Fuller test is the base of the augmented Dickey Fuller test, which we will discuss in the next section. The Dickey Fuller test can be described as follows.

For the following AR(1) process:

$$x_t = \alpha x_{t-1} + \epsilon_t$$

where ϵ_t is white noise

By doing the following manipulation, we can rewrite the above equation as follows:

$$\begin{aligned} x_t - x_{t-1} &= \alpha x_{t-1} - x_{t-1} + \epsilon_t \\ &= (\alpha - 1)x_{t-1} + \epsilon_t \end{aligned}$$

where $\gamma = (\alpha - 1)$

Hence, the basic Dickey Fuller test can be set up as follows:

$$H_0: \gamma = 0 \quad H_1: \gamma \neq 0$$

The above Dickey Fuller test is equivalent to test if $\alpha = 1$. However, under H_0 , the time series is not stationary and the usual normal distribution tests do not apply. The test statistic we will use is known as a unit root test statistic or Dickey Fuller (DF) statistic. When you run a Dickey Fuller test with statistical software, it will usually provide you with the critical test values at 1%, 5%, and 10% levels. By using the two-sided hypothesis test, if you fail to reject H_0 , this means there is a unit root for x_t . We will talk more about this when we demonstrate the test with an example.

There are three main versions of Dickey Fuller tests:

$$1) \nabla x_t = \gamma x_{t-1} + \epsilon_t$$

The test can be written as follows:

$$(\tau_1): \begin{cases} H_0: \gamma = 0 \\ H_1: \gamma \neq 0 \end{cases}$$

We just discussed this version of the test above. Under H_0 , this test is also used to test if the process is random walk. We label this test as τ_1 following Dickey and Fuller's article (Dickey and Fuller, 1979). Since there will be more versions of the Dickey Fuller test introduced in this section, it is easier to reference different versions of the test with their labels. The statistical software we will use later to demonstrate our example will also use the same labeling system for different versions of Dickey Fuller test.

$$2) \nabla x_t = \gamma x_{t-1} + \alpha_0 + \epsilon_t$$

This version of the equation adds a constant α_0 to the model. This equation is the same as random walk with a drift. As there are unit roots and constants to test, we can apply two versions of the test as follows:

$$\begin{aligned} a) (\phi_1): & \begin{cases} H_0: \gamma = 0, \alpha_0 = 0 \\ H_1: H_0 \text{ is not true} \end{cases} \\ b) (\tau_2): & \begin{cases} H_0: \gamma = 0 \\ H_1: H_0 \text{ is not true} \end{cases} \end{aligned}$$

$$3) \nabla x_t = \gamma x_{t-1} + \alpha_0 + \alpha_2 t + \varepsilon_t$$

This version of the equation adds a constant α_0 and a time trend t to the model. As there are unit roots, constants, and time trends to test, we can apply three versions of the test as follows:

- a) ϕ_2 : $\begin{cases} H_0: \gamma = 0, \alpha_0 = 0, \alpha_2 = 0 \\ H_1: H_0 \text{ is not true} \end{cases}$
- b) ϕ_3 : $\begin{cases} H_0: \gamma = 0, \alpha_0 = 0 \\ H_1: H_0 \text{ is not true} \end{cases}$
- c) τ_3 : $\begin{cases} H_0: \gamma = 0 \\ H_1: H_0 \text{ is not true} \end{cases}$

We can see each version of the test has some restrictions on the parameters of interest. For example, τ_1 , τ_2 , and τ_3 test for the presence of a unit root. ϕ_1 and ϕ_3 are to test for the presence of a unit root and the absence of a constant. ϕ_2 is to test the presence of unit roots and the absence of constants and time trends. If we are not sure about the model specifications, we would usually run all of them and compare the test results.

2.3 Augmented Dickey Fuller Test

The augmented Dickey Fuller test (ADF) is the most popular unit root test to use in research. Compared to the Dickey Fuller test, the ADF test includes the lag difference terms of the time series in the equation. The reason the ADF test includes lag difference terms is to control the serial correlation in the time series of interest. We can write the three versions of ADF equations and tests as follows (we will omit H_1 here as they are the same as in the last section):

$$1. \nabla x_t = \gamma x_{t-1} + \sum_{i=2}^p \beta_i \nabla x_{t-i+1} + \varepsilon_t$$

$$(\tau_1): H_0: \gamma = 0$$

$$2. \nabla x_t = \gamma x_{t-1} + \alpha_0 + \sum_{i=2}^p \beta_i \nabla x_{t-i+1} + \varepsilon_t$$

$$(\phi_1): H_0: \gamma = 0, \alpha_0 = 0$$

$$(\tau_2): H_0: \gamma = 0$$

$$3. \nabla x_t = \gamma x_{t-1} + \alpha_0 + \alpha_2 t + \sum_{i=2}^p \beta_i \nabla x_{t-i+1} + \varepsilon_t$$

$$(\phi_2): H_0: \gamma = 0, \alpha_0 = 0, \alpha_2 = 0$$

$$(\phi_3): H_0: \gamma = 0, \alpha_0 = 0$$

$$H_0: \gamma = 0$$

The versions of the ADF test are similar to the Dickey Fuller test, except that now we need to find the optimal lag order p for the equation. The software we use will automatically search for the optimal p based on the information criteria (AIC or BIC) we select for the test.

2.4 Kwiatkowski-Phillips-Schmidt-Shin (KPSS) Test

The KPSS test is a popular stationary test. It is usually used as a complement to unit root tests. The KPSS test determines whether there is a time trend (or deterministic trend) in the time series. In the beginning of the lesson, we learned a time series with only a time trend and white noise is trend stationary. Hence, KPSS can be described as a test if a time series is trend stationary. The KPSS test can be stated as follows:

- H_0 : the time series is trend stationary
- H_1 : the time series is not trend stationary

We can see that H_0 for the KPSS test and the ADF test show different hypotheses for the status of stationarity. H_0 for KPSS states that the time series is stationary while H_0 for ADF states that there is a unit root, so the time series is not stationary. Hence, be careful in reading the test results from different tests.

We can run both an ADF test and KPSS test to better understand the behavior of the time series.

- a. If the ADF test cannot reject H_0 and the KPSS test rejects H_0
 $\Rightarrow$, then the time series is non-stationary.
- b. If the ADF test rejects H_0 and the KPSS test cannot reject H_0
 $\Rightarrow$, then the time series is trend stationary.
- c. For other results, we will need to do more investigation.

2.5 After Unit Root Test

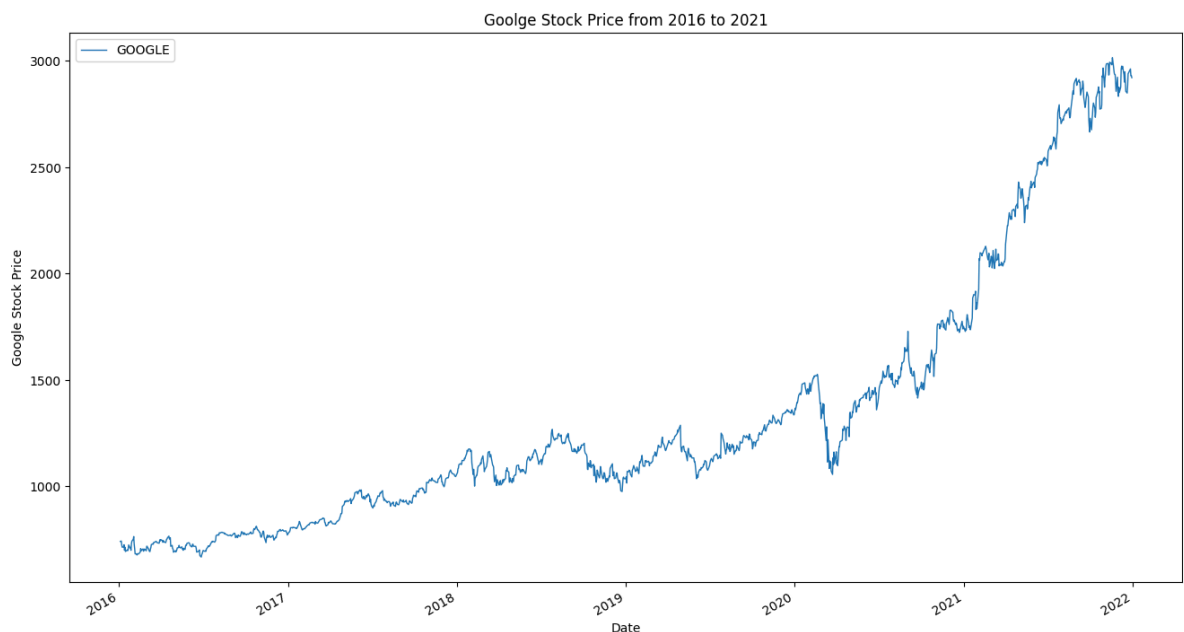
Once we conclude that a time series is non-stationary after a unit root test or a stationary test, we can take the necessary steps to make it stationary. If a time series has unit roots, we can take the first difference of the time series and then check the new time series again with a unit root test. It is an iterative process until the time series has no unit root and is stationary. Also, if there is a time trend, we can implement the regression method to remove the trend. Once we are sure the time series is stationary, we can move on to build an ARMA model or conduct the next analysis.

3. Unit Root Test Application: Google Stock Price

In this section, we are going to look into Google's stock price and run both ADF and KPSS tests on it. The time range of the time series is from 2016 to 2021. Let's look at the plot of Google's stock price first.

Figure 1: Google Stock Price Plot from 2016 to 2021

```
In [3]: # Plot Google price time series chart
goog.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="Google Stock Price",
    title="Goolge Stock Price from 2016 to 2021",
)
plt.show()
```



From the above Google price chart, we can see that Google's stock price grows at an exponential rate. Let's do three versions of the ADF and KPSS tests and see the results.

```
In [4]: # Create an customized ADF test function similar to ut.df function from R
def ADF_test(data, test_type='none', maxlag=None, autolag='BIC'):
    """
    Perform Dickey-Fuller/Augmented Dickey-Fuller test with all relevant test stati
    following the theoretical framework from Dickey and Fuller (1979)

    Parameters:
    - data: time series data (pandas Series or DataFrame)
    - test_type: 'none' (no constant, no trend), 'drift' (constant), 'trend' (const
    - maxlag: maximum number of lags to test
    - autolag: method for automatic lag selection ('BIC', 'AIC', 't-stat')

    The three main versions:
    1)  $\nabla x_t = \gamma x_{t-1} + u_t$  (test_type='none')
    2)  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + u_t$  (test_type='drift')
```

```

3)  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + \alpha_2 t + u_t$  (test_type='trend')
"""

# Clean and prepare data
if hasattr(data, 'dropna'):
    clean_data = data.dropna()
else:
    clean_data = pd.Series(data).dropna()

# Extract values as 1D array
if hasattr(clean_data, 'values'):
    y = clean_data.values.flatten()
else:
    y = np.array(clean_data).flatten()

# Map test_type to statsmodels regression parameter
regression_map = {
    'none': 'n',      # no constant, no trend
    'drift': 'c',     # constant only
    'trend': 'ct'     # constant and trend
}

regression = regression_map[test_type]

# Basic ADF test using statsmodels
try:
    adf_result = adfuller(y, regression=regression, maxlag=maxlag, autolag=auto
    lags_used = adf_result[2]

    # Create basic result structure
    result = {
        'test_type': test_type,
        'model_equation': get_model_equation(test_type),
        'adf_statistic': adf_result[0],
        'adf_pvalue': adf_result[1],
        'lags_used': lags_used,
        'n_observations': adf_result[3],
        'critical_values': adf_result[4],
        'ic_best': adf_result[5] if len(adf_result) > 5 else None
    }

    # Try to add enhanced regression results and test statistics
    try:
        enhanced_results = calculate_enhanced_statistics(y, test_type, lags_use
        result.update(enhanced_results)
    except Exception as e:
        # If enhanced calculation fails, just use basic ADF results
        result['enhancement_error'] = str(e)

    return result

except Exception as e:
    return {
        'test_type': test_type,
        'error': f"ADF test failed: {str(e)}"
    }

```



```

def calculate_enhanced_statistics(y, test_type, lags_used):
    """Calculate enhanced test statistics (tau and phi) with proper regression"""

    # Create first differences and lagged level
    dy = np.diff(y)
    y_lag1 = y[:-1]

    # Handle lagged differences for ADF extension
    if lags_used > 0:
        # Create lagged differences
        lagged_diffs = []
        for lag in range(1, lags_used + 1):
            lagged_diffs.append(dy[:-lag])

        # Align all arrays to same length
        min_length = len(dy) - lags_used
        dy_final = dy[lags_used:][:min_length]
        y_lag1_final = y_lag1[lags_used:][:min_length]

        # Stack lagged differences and trim to match
        if lagged_diffs:
            lagged_dy = np.column_stack([ld[-min_length:] for ld in lagged_diffs])
        else:
            lagged_dy = None
    else:
        dy_final = dy
        y_lag1_final = y_lag1
        lagged_dy = None

    # Build regression matrix
    if test_type == 'none':
        # Model:  $\nabla x_t = \gamma x_{t-1} + u_t$ 
        if lagged_dy is not None:
            X = np.column_stack([y_lag1_final, lagged_dy])
            var_names = ['gamma'] + [f'dy_lag{i+1}' for i in range(lags_used)]
        else:
            X = y_lag1_final.reshape(-1, 1)
            var_names = ['gamma']

    elif test_type == 'drift':
        # Model:  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + u_t$ 
        if lagged_dy is not None:
            X = np.column_stack([np.ones(len(dy_final)), y_lag1_final, lagged_dy])
            var_names = ['alpha_0', 'gamma'] + [f'dy_lag{i+1}' for i in range(lags_
        else:
            X = np.column_stack([np.ones(len(dy_final)), y_lag1_final])
            var_names = ['alpha_0', 'gamma']

    elif test_type == 'trend':
        # Model:  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + \alpha_2 t + u_t$ 
        trend = np.arange(1, len(dy_final) + 1)
        if lagged_dy is not None:
            X = np.column_stack([np.ones(len(dy_final)), trend, y_lag1_final, lagge
            var_names = ['alpha_0', 'alpha_2', 'gamma'] + [f'dy_lag{i+1}' for i in
        else:

```

```

X = np.column_stack([np.ones(len(dy_final)), trend, y_lag1_final])
var_names = ['alpha_0', 'alpha_2', 'gamma']

# Run OLS regression
model = OLS(dy_final, X).fit()

# Extract basic regression results
coefficients = dict(zip(var_names, model.params))
std_errors = dict(zip(var_names, model.bse))
t_statistics = dict(zip(var_names, model.tvalues))
p_values = dict(zip(var_names, model.pvalues))

# Calculate test statistics
test_statistics = {}

if test_type == 'none':
    #  $\tau_1$  test:  $H_0: \gamma = 0$ 
    gamma_t = t_statistics['gamma']
    test_statistics['tau1'] = {
        'statistic': gamma_t,
        'critical_value_5pct': -1.95, # Approximate 5% critical value for  $\tau_1$ 
        'null_hypothesis': ' $\gamma = 0$  (unit root exists)',
        'alternative_hypothesis': ' $\gamma \neq 0$  (stationary)',
        'interpretation': 'Tests for unit root in random walk model'
    }

elif test_type == 'drift':
    #  $\tau_2$  test:  $H_0: \gamma = 0$ 
    gamma_t = t_statistics['gamma']
    test_statistics['tau2'] = {
        'statistic': gamma_t,
        'critical_value_5pct': -2.86, # Approximate 5% critical value for  $\tau_2$ 
        'null_hypothesis': ' $\gamma = 0$  (unit root exists)',
        'alternative_hypothesis': ' $\gamma \neq 0$  (stationary)',
        'interpretation': 'Tests for unit root in random walk with drift model'
    }

#  $\phi_1$  test:  $H_0: \gamma = 0$  AND  $\alpha_0 = 0$ 
try:
    R_matrix = np.array([[1, 0], [0, 1]]) # Test first two coefficients
    if lagged_dy is not None:
        R_full = np.zeros((2, len(coefficients)))
        R_full[:, 2:] = R_matrix
        R_matrix = R_full
    phi1_test = model.f_test(R_matrix)
    test_statistics['phi1'] = {
        'statistic': phi1_test.fvalue,
        'critical_value_5pct': 3.17,
        'null_hypothesis': ' $\gamma = 0$  AND  $\alpha_0 = 0$  (unit root AND no drift)',
        'alternative_hypothesis': ' $\gamma \neq 0$  OR  $\alpha_0 \neq 0$  (stationary OR drift exi',
        'interpretation': 'Joint test for unit root and absence of constant'
    }
except:
    test_statistics['phi1'] = {
        'statistic': np.nan,
        'error': 'F-test calculation failed'
    }

```

```

    }

elif test_type == 'trend':
    #  $\tau_3$  test:  $H_0: \gamma = 0$ 
    gamma_t = t_statistics['gamma']
    test_statistics['tau3'] = {
        'statistic': gamma_t,
        'critical_value_5pct': -3.41, # Approximate 5% critical value for  $\tau_3$ 
        'null_hypothesis': ' $\gamma = 0$  (unit root exists)',
        'alternative_hypothesis': ' $\gamma \neq 0$  (stationary)',
        'interpretation': 'Tests for unit root in model with constant and trend'
    }

    #  $\phi_2$  test:  $H_0: \gamma = 0$  AND  $\alpha_0 = 0$  AND  $\alpha_2 = 0$ 
    try:
        R_matrix = np.eye(3) # Test first three coefficients
        if lagged_dy is not None:
            R_full = np.zeros((3, len(coefficients)))
            R_full[:3, :3] = R_matrix
            R_matrix = R_full
        phi2_test = model.f_test(R_matrix)
        test_statistics['phi2'] = {
            'statistic': phi2_test.fvalue,
            'critical_value_5pct': 4.68,
            'null_hypothesis': ' $\gamma = 0$  AND  $\alpha_0 = 0$  AND  $\alpha_2 = 0$  (unit root, no cons',
            'alternative_hypothesis': 'At least one parameter  $\neq 0$ ',
            'interpretation': 'Joint test for unit root and absence of constant'
        }
    except:
        test_statistics['phi2'] = {
            'statistic': np.nan,
            'error': 'F-test calculation failed'
        }

    #  $\phi_3$  test:  $H_0: \gamma = 0$  AND  $\alpha_0 = 0$ 
    try:
        R_matrix = np.array([[1, 0, 0], [0, 0, 1]]) # Test constant and gamma
        if lagged_dy is not None:
            R_full = np.zeros((2, len(coefficients)))
            R_full[0, 0] = 1 #  $\alpha_0$ 
            R_full[1, 2] = 1 # gamma
            R_matrix = R_full
        phi3_test = model.f_test(R_matrix)
        test_statistics['phi3'] = {
            'statistic': phi3_test.fvalue,
            'critical_value_5pct': 3.17,
            'null_hypothesis': ' $\gamma = 0$  AND  $\alpha_0 = 0$  (unit root AND no constant)',
            'alternative_hypothesis': ' $\gamma \neq 0$  OR  $\alpha_0 \neq 0$  (stationary OR constant',
            'interpretation': 'Joint test for unit root and absence of constant'
        }
    except:
        test_statistics['phi3'] = {
            'statistic': np.nan,
            'error': 'F-test calculation failed'
        }

```

```

return {
    'test_statistics': test_statistics,
    'regression_results': {
        'coefficients': coefficients,
        'std_errors': std_errors,
        't_statistics': t_statistics,
        'p_values': p_values
    },
    'model_fit': {
        'r_squared': model.rsquared,
        'r_squared_adj': model.rsquared_adj,
        'fstatistic': model.fvalue,
        'f_pvalue': model.f_pvalue
    }
}

def get_model_equation(test_type):
    """Return the model equation string for display"""
    equations = {
        'none': '∇x_t = γx_{t-1} + dy_lag(s) + u_t',
        'drift': '∇x_t = γx_{t-1} + α_0 + dy_lag(s) + u_t',
        'trend': '∇x_t = γx_{t-1} + α_0 + α_2*t + dy_lag(s) + u_t'
    }
    return equations[test_type]

def print_dickey_fuller_summary(result):
    """Print comprehensive Dickey-Fuller test results"""

    print(f"\n{'='*80}")
    print(f"AUGMENTED-DICKEY-FULLER TEST RESULTS - {result['test_type'].upper()} MO")
    print(f"{'='*80}")

    # Check for errors
    if 'error' in result:
        print(f"ERROR: {result['error']}")
        return

    # Basic information
    print(f"Model: {result.get('model_equation', 'N/A')}")
    print(f"Lags Used: {result.get('lags_used', 'N/A')}")
    print(f"Number of Observations: {result.get('n_observations', 'N/A')}")
    if result.get('ic_best'):
        print(f"BIC: {result['ic_best']:.6f}")

    # Regression results
    if 'regression_results' in result:
        print(f"\n{'='*60}")
        print("REGRESSION RESULTS")
        print(f"{'='*60}")

        reg = result['regression_results']

        print(f"{'Parameter':<12} {'Coefficient':<12} {'Std Error':<12} {'t-statist")
        print(f"{'-' * 68}")

        for param in reg['coefficients'].keys():

```

```

coef = reg['coefficients'][param]
se = reg['std_errors'][param]
t_stat = reg['t_statistics'][param]
p_val = reg['p_values'][param]

marker = " <  $\gamma$  (unit root)" if param == 'gamma' else ""
print(f"{param:<12} {coef:<12.6f} {se:<12.6f} {t_stat:<12.6f} {p_val:<12.6f}")

# Model fit
if 'model_fit' in result:
    fit = result['model_fit']
    print(f"\nModel Fit:")
    print(f"   R-squared: {fit['r_squared']:.6f}")
    print(f"   Adjusted R-squared: {fit['r_squared_adj']:.6f}")
    print(f"   F-statistic: {fit['fstatistic']:.6f}")

# Enhanced test statistics
if 'test_statistics' in result:
    print(f"\n{'='*60}")
    print("DETAILED TEST STATISTICS")
    print(f"\n{'='*60}")

    for test_name, test_info in result['test_statistics'].items():
        print(f"\n{test_name.upper()} Test:")

        if 'error' in test_info:
            print(f"   Error: {test_info['error']}")
            continue

        print(f"   H0: {test_info['null_hypothesis']}")
        print(f"   H1: {test_info['alternative_hypothesis']}")

        stat_val = test_info['statistic']
        print(f"   Test Statistic: {stat_val:.6f}")

        # Handle critical values for both tau and phi tests
        if 'critical_value_5pct' in test_info:
            crit_val = test_info['critical_value_5pct']
            print(f"   5% Critical Value: {crit_val}")

        # Decision logic for tau tests (left-tail test)
        if 'tau' in test_name:
            if stat_val < crit_val:
                print(f"   Decision: Reject H0 (t-stat < critical value)")
            else:
                print(f"   Decision: Fail to reject H0 (t-stat > critical value)")

        # Decision logic for phi tests (right-tail test)
        elif 'phi' in test_name:
            if stat_val > crit_val:
                print(f"   Decision: Reject H0 (F-stat > critical value)")

```

```

else:
    print(f" Decision: Fail to reject H0 (F-stat < critical va

```

Figure 2: Google Stock Price, ADF Test without Drift and Trend

```

In [5]: # Google Stock Price, ADF Test without Drift and Trend
df_goog_none = ADF_test(goog, test_type='none', autolag='BIC')
print_dickey_fuller_summary(df_goog_none)

=====
AUGMENTED-DICKEY-FULLER TEST RESULTS - NONE MODEL
=====
Model:  $\nabla x_t = \gamma x_{t-1} + \text{dy\_lag}(s) + u_t$ 
Lags Used: 1
Number of Observations: 1503
BIC: 13508.807192

=====
REGRESSION RESULTS
=====
Parameter      Coefficient    Std Error      t-statistic    p-value
-----
gamma           0.001248       0.000410       3.047900       0.002345    ←  $\gamma$  (unit root)
dy_lag1        -0.089269      0.025744      -3.467602      0.000540

Model Fit:
  R-squared: 0.012903
  Adjusted R-squared: 0.011588
  F-statistic: 9.810128

=====
DETAILED TEST STATISTICS
=====

TAU1 Test:
  H0:  $\gamma = 0$  (unit root exists)
  H1:  $\gamma \neq 0$  (stationary)
  Test Statistic: 3.047900
  5% Critical Value: -1.95
  Decision: Fail to reject H0 (t-stat > critical value)

```

From figure 2, we see that the ADF test's statistic is 3.048. The 5% critical value is -1.95. Hence, we cannot reject H_0 . The conclusion is that there is a unit root in Google's stock price.

Now let's run an ADF test with drift and see the results.

Figure 3: Google Stock Price, ADF Test with Drift but without Trend

```

In [6]: # ADF test with drift but no trend
df_goog_drift = ADF_test(goog, test_type='drift', autolag='BIC')
print_dickey_fuller_summary(df_goog_drift)

```

```
=====
AUGMENTED-DICKEY-FULLER TEST RESULTS - DRIFT MODEL
=====
Model:  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + \nabla y_{lag}(s) + u_t$ 
Lags Used: 1
Number of Observations: 1503
BIC: 13516.069302
```

```
=====
REGRESSION RESULTS
=====
Parameter      Coefficient      Std Error      t-statistic      p-value
-----
alpha_0         -0.454600         1.441896        -0.315279         0.752594
gamma            0.001533          0.000992         1.545548         0.122425      ← γ (unit root)
dy_lag1         -0.089597         0.025773        -3.476457         0.000523

Model Fit:
  R-squared: 0.009094
  Adjusted R-squared: 0.007773
  F-statistic: 6.882952
```

```
=====
DETAILED TEST STATISTICS
=====
```

```
TAU2 Test:
  H0: γ = 0 (unit root exists)
  H1: γ ≠ 0 (stationary)
  Test Statistic: 1.545548
  5% Critical Value: -2.86
  Decision: Fail to reject H0 (t-stat > critical value)
```

```
PHI1 Test:
  H0: γ = 0 AND α0 = 0 (unit root AND no drift)
  H1: γ ≠ 0 OR α0 ≠ 0 (stationary OR drift exists)
  Test Statistic: 4.691762
  5% Critical Value: 3.17
  Decision: Reject H0 (F-stat > critical value)
```

From Figure 3, we can see there are two model results: τ_2 and ϕ_1 . For model τ_2 , the ADF test statistics is 1.5455 and the 5% critical value is -2.86. Hence, we cannot reject H_0 that there is a unit root. For model ϕ_1 , the ADF test statistics is 4.6918 and the 5% critical value is 3.17. Hence, we reject the H_0 that the drift is 0. The conclusion is that we have a unit root and a drift in Google stock price.

Next let's run the ADF test with both drift and trend.

Figure 4: Google Stock Price, ADF Test with Drift and Trend

```
In [7]: # ADF test with drift and trend
df_goog_trend = ADF_test(goog, test_type='trend', autolag='BIC')
print_dickey_fuller_summary(df_goog_trend)
```

```

=====
AUGMENTED-DICKEY-FULLER TEST RESULTS - TREND MODEL
=====
Model:  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + \alpha_2 t + dy_{lag}(s) + u_t$ 
Lags Used: 1
Number of Observations: 1503
BIC: 13521.205277

=====
REGRESSION RESULTS
=====
Parameter      Coefficient      Std Error      t-statistic      p-value
-----
alpha_0         -0.054849        1.462471       -0.037504        0.970088
alpha_2         0.004558         0.002838       1.606160        0.108449
gamma           -0.001353        0.002052       -0.659378        0.509754    ←  $\gamma$  (unit root)
dy_lag1         -0.088493        0.025768       -3.434217        0.000610

Model Fit:
  R-squared: 0.010796
  Adjusted R-squared: 0.008816
  F-statistic: 5.453384

=====
DETAILED TEST STATISTICS
=====

TAU3 Test:
  H0:  $\gamma = 0$  (unit root exists)
  H1:  $\gamma \neq 0$  (stationary)
  Test Statistic: -0.659378
  5% Critical Value: -3.41
  Decision: Fail to reject H0 (t-stat > critical value)

PHI2 Test:
  H0:  $\gamma = 0$  AND  $\alpha_0 = 0$  AND  $\alpha_2 = 0$  (unit root, no constant, no trend)
  H1: At least one parameter  $\neq 0$ 
  Test Statistic: 3.991052
  5% Critical Value: 4.68
  Decision: Fail to reject H0 (F-stat < critical value)

PHI3 Test:
  H0:  $\gamma = 0$  AND  $\alpha_0 = 0$  (unit root AND no constant)
  H1:  $\gamma \neq 0$  OR  $\alpha_0 \neq 0$  (stationary OR constant exists)
  Test Statistic: 0.351912
  5% Critical Value: 3.17
  Decision: Fail to reject H0 (F-stat < critical value)

```

Figure 4 shows the ADF test results for a model with drift and trend. We see that we cannot reject H_0 at a 5% critical value level for all three models. This means that the time trend should not be included in the model. Let's run a KPSS test to see if we get the same result.

Figure 5: Google Stock Price, KPSS Test Result


```
In [8]: # KPSS test
# (Set lags=-1 to use the old method that only depends on the sample size)
print(KPSS(goog, trend="ct", lags=-1))
```

```

      KPSS Stationarity Test Results
=====
Test Statistic           1.056
P-value                  0.000
Lags                     24
-----

Trend: Constant and Linear Time Trend
Critical Values: 0.22 (1%), 0.15 (5%), 0.12 (10%)
Null Hypothesis: The process is weakly stationary.
Alternative Hypothesis: The process contains a unit root.
```

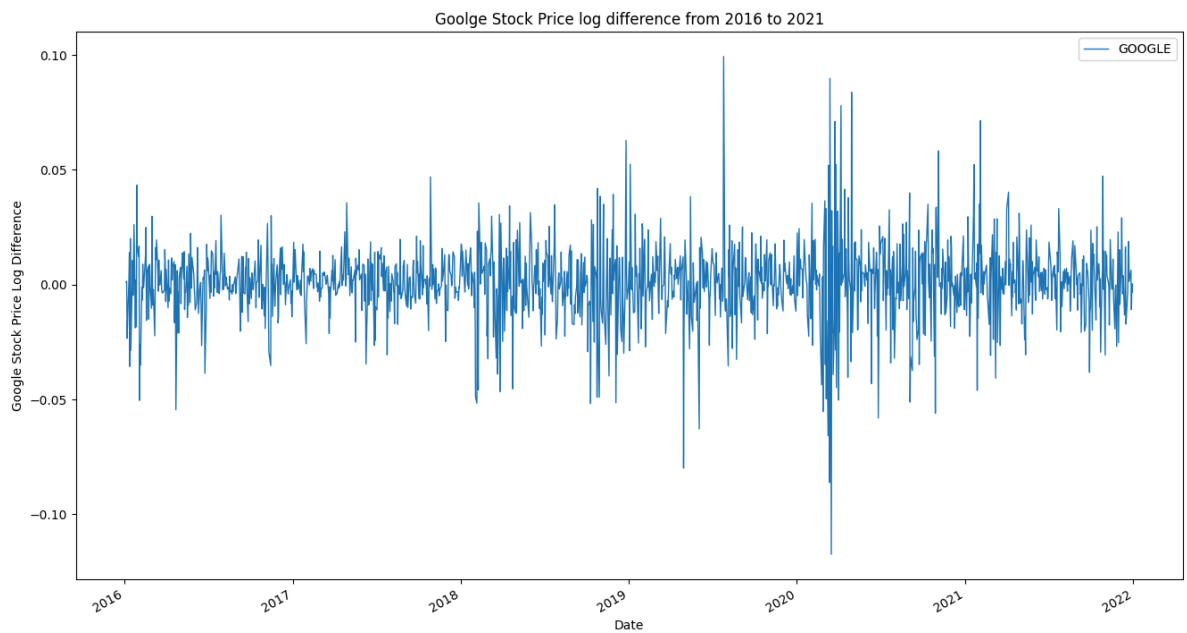
In figure 5, we can see the test results show that the p -value is smaller than 5%. It is a clear sign to reject H_0 . The conclusion is Google's stock price is not trend stationary and has a unit root.

From the above analysis, we can see Google's stock price follows a random walk with a drift model.

Let's do the log difference transformation we mentioned in the previous module. Log difference transformation approximates to a stock return. Figure 2 shows the plot for Google's stock price with log difference transformation.

Figure 6: Plot of Logged Difference of Google Stock Price from 2016 to 2021

```
In [9]: # Plot Google price time series chart
lgoog = np.log(goog).diff().dropna()
lgoog.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="Google Stock Price Log Difference",
    title="Google Stock Price log difference from 2016 to 2021",
)
plt.show()
```



Now let's run the three versions of an ADF test for the logged difference of Google's stock price and see the results. We are only going to focus on the ADF test statistics and 5% critical values for the three models. The following table only provides the information necessary from the model results to make a decision.

Figure 7: Three Versions of ADF Test for Logged Difference of Google Stock Price

```
In [10]: # Logged Difference of Google Stock Price, ADF Test without Drift and Trend
df_lgoog_none = ADF_test(lgoog, test_type='none', autolag='BIC')
print_dickey_fuller_summary(df_lgoog_none)
```

```
=====
AUGMENTED-DICKEY-FULLER TEST RESULTS - NONE MODEL
=====
```

```
Model:  $\nabla x_t = \gamma x_{t-1} + \Delta y_{lag}(s) + u_t$ 
Lags Used: 0
Number of Observations: 1503
BIC: -7992.618005
```

```
=====
REGRESSION RESULTS
=====
```

Parameter	Coefficient	Std Error	t-statistic	p-value	
gamma	-1.112520	0.025639	-43.391403	0.000000	← γ (unit root)

```
Model Fit:
R-squared: 0.556253
Adjusted R-squared: 0.555958
F-statistic: 1882.813856
```

```
=====
DETAILED TEST STATISTICS
=====
```

```
TAU1 Test:
H0:  $\gamma = 0$  (unit root exists)
H1:  $\gamma \neq 0$  (stationary)
Test Statistic: -43.391403
5% Critical Value: -1.95
Decision: Reject H0 (t-stat < critical value)
```

```
In [11]: # Logged Difference of Google Stock Price, ADF Test with Drift but without Trend
df_lgoog_drift = ADF_test(lgoog, test_type='drift', autolag='BIC')
print_dickey_fuller_summary(df_lgoog_drift)
```

```
=====
AUGMENTED-DICKEY-FULLER TEST RESULTS - DRIFT MODEL
=====
```

Model: $\nabla x_t = \gamma x_{t-1} + \alpha_0 + dy_{lag}(s) + u_t$
 Lags Used: 0
 Number of Observations: 1503
 BIC: -7992.196100

```
=====
REGRESSION RESULTS
=====
```

Parameter	Coefficient	Std Error	t-statistic	p-value	
alpha_0	0.001017	0.000421	2.416234	0.015801	
gamma	-1.115968	0.025638	-43.528345	0.000000	← γ (unit root)

Model Fit:
 R-squared: 0.557973
 Adjusted R-squared: 0.557678
 F-statistic: 1894.716841

```
=====
DETAILED TEST STATISTICS
=====
```

TAU2 Test:
 $H_0: \gamma = 0$ (unit root exists)
 $H_1: \gamma \neq 0$ (stationary)
 Test Statistic: -43.528345
 5% Critical Value: -2.86
 Decision: Reject H_0 (t-stat < critical value)

PHI1 Test:
 $H_0: \gamma = 0$ AND $\alpha_0 = 0$ (unit root AND no drift)
 $H_1: \gamma \neq 0$ OR $\alpha_0 \neq 0$ (stationary OR drift exists)
 Test Statistic: 947.358445
 5% Critical Value: 3.17
 Decision: Reject H_0 (F-stat > critical value)

```
In [12]: # Logged Difference of Google Stock Price, ADF Test with Drift and Trend
df_lgoog_trend = ADF_test(lgoog, test_type='trend', autolag='BIC')
print_dickey_fuller_summary(df_lgoog_trend)
```

```
=====
AUGMENTED-DICKEY-FULLER TEST RESULTS - TREND MODEL
=====
Model:  $\nabla x_t = \gamma x_{t-1} + \alpha_0 + \alpha_2 t + dy_{lag}(s) + u_t$ 
Lags Used: 0
Number of Observations: 1503
BIC: -7985.603481
```

```
=====
REGRESSION RESULTS
=====
Parameter      Coefficient      Std Error      t-statistic      p-value
-----
alpha_0         0.000178         0.000841         0.211640         0.832417
alpha_2         0.000001         0.000001         1.152640         0.249242
gamma           -1.116759         0.025644        -43.548365        0.000000      ←  $\gamma$  (unit root)
```

```
Model Fit:
  R-squared: 0.558364
  Adjusted R-squared: 0.557775
  F-statistic: 948.230094
```

```
=====
DETAILED TEST STATISTICS
=====
```

```
TAU3 Test:
  H0:  $\gamma = 0$  (unit root exists)
  H1:  $\gamma \neq 0$  (stationary)
  Test Statistic: -43.548365
  5% Critical Value: -3.41
  Decision: Reject H0 (t-stat < critical value)
```

```
PHI2 Test:
  H0:  $\gamma = 0$  AND  $\alpha_0 = 0$  AND  $\alpha_2 = 0$  (unit root, no constant, no trend)
  H1: At least one parameter  $\neq 0$ 
  Test Statistic: 632.153412
  5% Critical Value: 4.68
  Decision: Reject H0 (F-stat > critical value)
```

```
PHI3 Test:
  H0:  $\gamma = 0$  AND  $\alpha_0 = 0$  (unit root AND no constant)
  H1:  $\gamma \neq 0$  OR  $\alpha_0 \neq 0$  (stationary OR constant exists)
  Test Statistic: 948.230048
  5% Critical Value: 3.17
  Decision: Reject H0 (F-stat > critical value)
```

From figure 7, we can see that there is no unit root for the logged difference of Google's stock price. Let's run a KPSS test for the logged difference of Google's stock price as well.

Figure 8: KPSS Test for Logged Difference of Google's Stock Price

```
In [13]: # KPSS test
# (Set lags=-1 to use the old method that only depends on the sample size)
print(KPSS(lgoog, trend="ct", lags=-1))
```

KPSS Stationarity Test Results	
=====	
Test Statistic	0.038
P-value	0.752
Lags	24

Trend: Constant and Linear Time Trend
Critical Values: 0.22 (1%), 0.15 (5%), 0.12 (10%)
Null Hypothesis: The process is weakly stationary.
Alternative Hypothesis: The process contains a unit root.

From figure 8, we can see that the KPSS test cannot reject H_0 , and we can conclude that there is no unit root. From the ADF and KPSS tests, we can say that the logged difference of Google's stock price is stationary.

4. Conclusion

In this lesson, we introduced a unit root test to test for stationarity of a time series. We first described what a unit root is. We then introduced the first unit root test: the Dickey Fuller Test. We went through the basic theory and the three versions of the test. Although we do not use Dickey Fuller test as much now, it serves as a good base test to understand the next test: the augmented Dickey Fuller test. ADF is the most popular unit root test nowadays. We also went through its theory and the three versions of the test. We then discussed the KPSS test. This is another unit root test that can be used to compliment the ADF test. Finally, we used Google's stock price as an example to run through the ADF test and KPSS test. We showed how to read the results from the tests and draw conclusions. This lesson concludes our study of analysis of univariate time series or one time series. In the next few lessons, we will start to study two or more time series and analyze their interactions.

References

- Dickey, David A., and Wayne A. Fuller. "Distribution of the Estimator for Autoregressive Time Series with a Unit Root." Journal of the American Statistical Association, vol. 74, no. 366, 1979, pp. 427–431.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. PennState, Eberly College of Science. "11.2: Vector Autoregressive Models VAR(p) Models." STAT 510: Applied Time Series Analysis. Lecture. <https://online.stat.psu.edu/stat510/lesson/11/11.2>

- **Required pages:** Entire webpage
- **Estimated reading time:** 2 hours

Lesson 2 - Ergodicity and Vector Autoregressive Model

December 23, 2025

0.1 FINANCIAL ECONOMETRICS

MODULE 6 | LESSON 2

1 ERGODICITY AND VECTOR AUTOREGRESSIVE MODEL

Reading Time	60 minutes
Prior Knowledge	Basic Time Series
Keywords	Ergodicity, Vector Autoregressive Model, Contemporaneous correlation, Seemingly Unrelated Regression

—All rights reserved WQU WorldQuant University QQQQ

So far, we have been learning how to analyze single time series or univariate time series. Starting from this lesson, we are going to expand our analysis to two time series or bivariate time series. The theory can easily extend to multiple time series. The first multivariate time series model we are going to look into is the vector autoregressive model (VAR). Then, we will present an example of how to apply the VAR model to forecast financial asset performance. Before talking about the VAR model, we will cover a desired property we would like to have for a time series: ergodicity.

```
[1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from arch.unitroot import ADF
from pandas.tseries.holiday import USFederalHolidayCalendar
from pandas.tseries.offsets import CustomBusinessDay
from statsmodels.tsa.api import VAR

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

```
[2]: # Download datasets
m6_data = pd.read_csv("M6. goog_eur_10.csv")

# Convert date variable to date format and set index
m6_data["Date2"] = pd.to_datetime(m6_data["Date"], format="%m/%d/%Y")
```



```
data_set = m6_data.loc[:,].set_index("Date2")

# Selecting GOOGLE, EURUSD, UST10Y
goog = data_set.GOOGLE
eur = data_set.EURUSD
ust10 = data_set.UST10Y
```

1.1 1. Ergodicity

In Module 4, Lesson 1, we explained what a time series is. Let's do a quick review. Assume we have the price of Stock A for five days as follows:

Table 1: Stock A Price for 5 Days

Date	Price: Observed Data	Associated Random Variable
day 1	x_1	X_1
day 2	x_2	X_2
day 3	x_3	X_3
day 4	x_4	X_4
day 5	x_5	X_5

From table 1, we see the Stock A price data points as $\{x_1, x_2, x_3, x_4, x_5\}$. In time series analysis, we say these data points are one data realization from a series of random variables $\{X_1, X_2, X_3, X_4, X_5\}$ in five days. Given this setup, Stock A price data points can have another realization, say $\{x'_1, x'_2, x'_3, x'_4, x'_5\}$, from the series of random variables over a series of equally distanced time points. We call the system of these series of random variables a **stochastic process**. AR, MA, or ARIMA models are all examples of stochastic processes. The time series data we actually obtain is just one data realization from the stochastic process. We can treat the time series data as just one sample from the stochastic process. If we do another sampling, we will get another collection of data points. In the real world, we only get one realization of data points though.

Even though we only get one data realization in the real world, we usually say the more data points we get from our sample, the better we are informed about the structure that generates these data points. This is the general idea of the Law of Large Numbers (LLN). The property of **ergodicity** is to ensure that a stochastic process can be properly established by a large number of data points under LLN. We can also say that as we get more data points, we will have more information about a stochastic process that generates the data points if this process is ergodic.

Okay, we know how the property of ergodicity of a stochastic process can help us understand the process. However, how can we define ergodicity in order to offer the property we just described?

The intuitive definition for **ergodicity** is as follows: A stochastic process $\{X_t\}_{t=-\infty}^{\infty}$ is ergodic if any two collection of random variables partitioned far apart in the sequence are essentially independent.

Let's use an example to explain the definition. Say we pick the random variables at $t = 1$ and $t = 3$, which will be X_1 and X_3 . Assume their correlation, $\text{corr}(X_1, X_3)$, is 0.6. Their time distance is $3 - 1 = 2$. Then, we pick another pair of random variables, say at $t = 23$ and $t = 45$. Assume their correlation, $\text{corr}(X_{23}, X_{45})$, is 0.3. Their time difference is $45 - 23 = 22$. Lets pick another

pair of random variables, say at $t = 11$ and $t = 1432$. Assume their correlation, $\text{corr}(X_{11}, X_{1432})$, is 0.001. Their time difference is $1432 - 11 = 1421$. From the above example of stochastic process X , we can see as the time difference gets larger, the correlation of two variables from X process get smaller. When this time difference gets large enough for any two random variables from process X , the correlation of the two random variables goes close to 0. This diminishing dependence of two random variables as their time difference gets wider illustrates the concept of ergodicity. According to ergodicity, if two random variables are far apart enough, their correlation will be very low. It also means they will provide different information about the stochastic process to which they belong. Hence, we can also say the property of ergodicity property will make sure LLN will work for the stochastic process.

If a time series x_t is stationary and $\lim_{j \rightarrow \infty} \text{cov}(x_t, x_{t-j}) = 0$, then x_t is ergodic for mean. It means that if we have a large enough data sample from time series x_t , the sample mean of the time series is a consistent estimate of the population mean of the underlying process.

1.2 2. Vector Autoregressive Model (VAR)

Vector Autoregressive Model (VAR) is a time series model that is used to forecast two or more time series. VAR model can also be used to understand the joint dynamic interaction among time series. VAR is a multiple variables version of an AR model. Let's use a VAR model to explain how it works.

Assume we have two time series x_t and y_t and we would like to understand how they interact with each other. With a lag 2 VAR model, we can write the model as follows:

$$x_t = \alpha_0 + \alpha_1 x_{t-1} + \alpha_2 x_{t-2} + \beta_1 y_{t-1} + \beta_2 y_{t-2} + e_t$$

$$y_t = \phi_0 + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \theta_1 y_{t-1} + \theta_2 y_{t-2} + \epsilon_t$$

Where $e_t \sim \text{white noise}(0, \sigma_e^2)$ and $\epsilon_t \sim \text{white noise}(0, \sigma_\epsilon^2)$ and both e_t and ϵ_t are not autocorrelated.

There are a few key features for a VAR model.

a. VAR model is not a simultaneous model.

One key point to note is that the current values of x_t and y_t only depend on the past values of both time series. This point is important because it means that a VAR model does not require solving both equations at the same time to estimate the parameters of the model. A VAR model is not a simultaneous model. We can actually estimate each equation separately. This is the key point why the VAR model is popular because this structure makes estimating the model and forecasting very convenient.

b. e_t and ϵ_t are contemporaneously correlated.

e_t and ϵ_t can be correlated. They can both be impacted by the factors that are not in the equation system. For example, if x_t and y_t are two stock prices, the outside factors like the outbreak of war or government action on financial markets can impact both e_t and ϵ_t at the same time. We call e_t and ϵ_t contemporaneously correlated. We also can call this equation system seemingly unrelated regression (SUR).

c. x_t and y_t are stationary and ergodic.

If x_t and y_t are $I(1)$, then we can take the first difference to both time series to make them stationary. A VAR model can actually use non-stationary time series directly in the model. However, the model will need to make some adjustments before estimation. We will talk more about this situation in the following lessons.

d. Estimate Each Equation Using OLS Estimation.

Like we mentioned in point A, each equation can be estimated separately because they are not simultaneous equations and they have identical exogenous variables. We can estimate each equation by using OLS estimation, one at a time.

In our example, we have two time series, two equations, and two lags in the equations. We call this VAR model a two-dimension VAR(2) model. Two dimension refers to the two time series in the model. VAR(2) refers to the two lags in the model. In the next section, we are going to show how to run a VAR model for financial assets.

1.3 3. VAR Model Application: Google Stock Price, EUR/USD Forex Rate and U.S. 10-Year Treasury Yield

In this section, we are going to use data from Google Stock Price, EUR/USD Forex Rate, and the U.S. 10-year Treasury yield to build a VAR model. First, let's draw time plots for the three time series.

Figure 1: Time Plots for Google Stock Price, EUR/USD Forex Rate and U.S. 10-Year Treasury Yield from 2016 to 2021

```
[3]: # Time plots for Google Stock Price, EUR/USD Forex Rate and U.S. 10-Year Treasury Yield
fig, axs = plt.subplots(2, 2)

goog.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="Google Stock Price",
    title="Google Stock Price from 2016 to 2021",
    ax=axs[0, 0],
)

eur.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="EURUSD Forex Rate",
    title="EURUSD Exchange Rate from 2016 to 2021",
    ax=axs[0, 1],
)

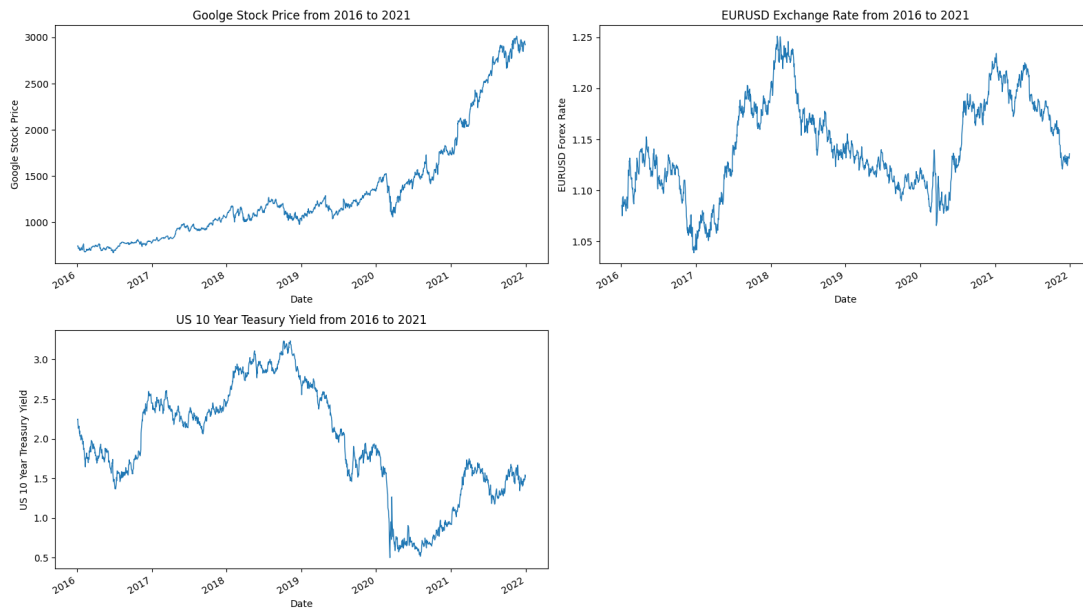
ust10.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="US 10 Year Treasury Yield",
```

```

title="US 10 Year Treasury Yield from 2016 to 2021",
ax=axes[1, 0],
)

axes[1, 1].axis("off")
fig.tight_layout()
plt.show()

```



From figure 1, we can see that Google's stock price has a long-term trend and the other two assets have short trends during the period. Let's ADF test to check for their stationarity.

Figure 2: ADF Test Results with 5% Significance Level for Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield

```

[4]: # ADF Test Results with 5% Significance Level for GOOGLE, EURUSD, UST10Y
goog_adf = ADF(goog, trend="n", method="bic")
eur_adf = ADF(eur, trend="n", method="bic")
ust10_adf = ADF(ust10, trend="n", method="bic")

pd.DataFrame(
    {
        "Google Stock Price": (goog_adf.stat, goog_adf.critical_values["5%"]),
        "| EURUSD Forex Rate": (eur_adf.stat, eur_adf.critical_values["5%"]),
        "| US 10 Year Treasury Yield": (
            ust10_adf.stat,
            ust10_adf.critical_values["5%"],
        ),
    },
)

```

```

    },
    index=["ADF Test Statistic", "5% Critical Value"],
)

```

```

[4]:
Google Stock Price | EURUSD Forex Rate \
ADF Test Statistic      3.04790      0.164388
5% Critical Value      -1.94118      -1.941180

      | US 10 Year Treasury Yield
ADF Test Statistic      -0.810331
5% Critical Value      -1.941180

```

From figure 2, we can see that none of the financial assets has an ADF test statistic lower than 5% critical value. Hence, we cannot reject H_0 hypothesis and there are unit roots in three time series. Let's take the first difference and check the differenced time series again.

Figure 3: Time Plots for Differenced Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield from 2016 to 2021

```

[5]: # Time Plots for Differenced GOOGLE, EURUSD, UST10Y
fig, axs = plt.subplots(2, 2)

lgoog = np.log(goog).diff().dropna()
lgoog.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="Google Stock Price Log Difference",
    title="Google Stock Price Log Difference from 2016 to 2021",
    ax=axs[0, 0],
)

leur = np.log(eur).diff().dropna()
leur.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="EURUSD Exchange Rate Log Difference",
    title="EURUSD Exchange Rate Log Difference from 2016 to 2021",
    ax=axs[0, 1],
)

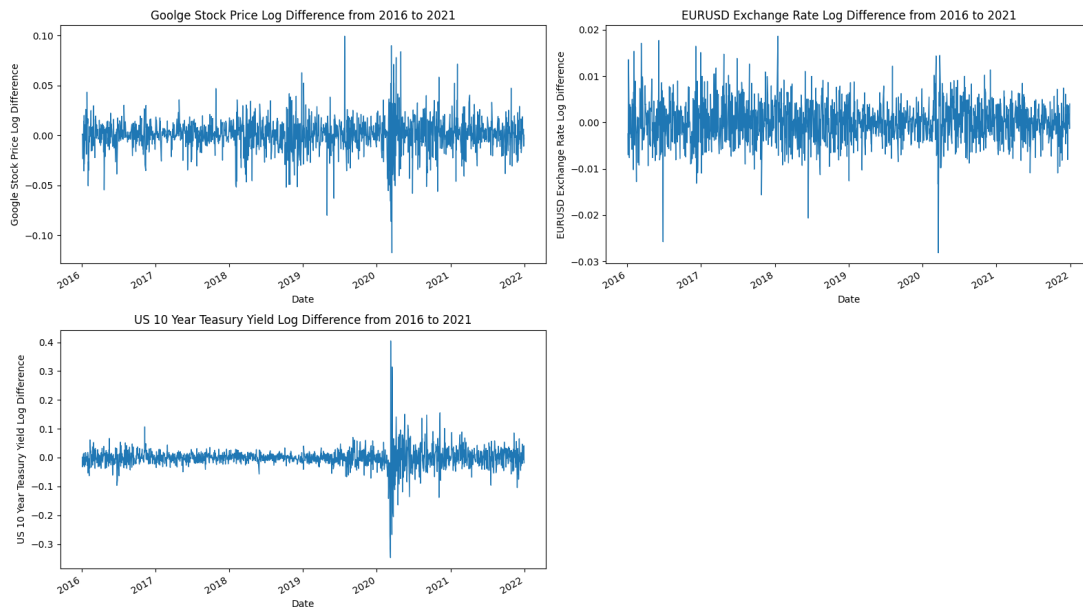
lust10 = np.log(ust10).diff().dropna()
lust10.plot(
    linewidth=1,
    xlabel="Date",
    ylabel="US 10 Year Treasury Yield Log Difference",
    title="US 10 Year Treasury Yield Log Difference from 2016 to 2021",
    ax=axs[1, 0],
)

```

```

axs[1, 1].axis("off")
fig.tight_layout()
plt.show()

```



From figure 3, we can see that the three differenced time series are all moving around 0 now. Let's check their ADF tests.

Figure 4: ADF Test Results with 5% Significance Level for Differenced Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield

```

[6]: # ADF Test Results with 5% Significance Level for Differenced GOOGLE, EURUSD,
      ↳ UST10Y
lgoog_adf = ADF(lgoog, trend="n", method="bic")
leur_adf = ADF(leur, trend="n", method="bic")
lust10_adf = ADF(lust10, trend="n", method="bic")

pd.DataFrame(
    {
        "Google Stock Price": (lgoog_adf.stat, lgoog_adf.critical_values["5%"]),
        "| EURUSD Forex Rate": (leur_adf.stat, leur_adf.critical_values["5%"]),
        "| US 10 Year Treasury Yield": (
            lust10_adf.stat,
            lust10_adf.critical_values["5%"],
        ),
    },
    index=["ADF Test Statistic", "5% Critical Value"],

```

```
)
```

```
[6]:          Google Stock Price | EURUSD Forex Rate \
ADF Test Statistic          -43.391403          -39.458187
5% Critical Value           -1.941180           -1.941180

          | US 10 Year Treasury Yield
ADF Test Statistic          -16.826455
5% Critical Value           -1.941181
```

From figure 4, we can see the ADF test statistics for three time series are all lower than the 5% critical value. Therefore, we can conclude that we can reject H_0 hypothesis and state that the three differenced time series do not have unit roots.

The next thing we need to do is to decide how many lags we would like to include in our VAR model for Google's stock price, EUR/USD Forex rate, and U.S. 10-year Treasury yield. The software helps us select the number of lags for VAR model using information criteria. Here is the result of selecting the optimal number of lags for the differences of three time series.

Figure 5: VAR Model Lag Selection for Differenced Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield

```
[7]: # VAR Model Lag Selection for Differenced GOOGLE, EURUSD, UST10Y

# Join log time series in one DataFrame
diff_data = pd.concat([lgoog, leur, lust10], axis=1)

# Fit VAR model and run lag selection tool
model = VAR(diff_data)
x = model.select_order(maxlags=12, trend="c")
x.summary()
```

```
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
    self._init_dates(dates, freq)
```

```
[7]:
```

	AIC	BIC	FPE	HQIC
0	-25.77	-25.76*	6.404e-12	-25.77
1	-25.80	-25.75	6.260e-12	-25.78*
2	-25.80	-25.72	6.247e-12	-25.77
3	-25.80	-25.70	6.228e-12	-25.76
4	-25.81	-25.67	6.197e-12	-25.76
5	-25.81	-25.64	6.151e-12	-25.75
6	-25.82	-25.62	6.112e-12	-25.75
7	-25.86	-25.63	5.858e-12	-25.78
8	-25.87	-25.60	5.825e-12	-25.77
9	-25.87*	-25.57	5.804e-12*	-25.76
10	-25.87	-25.54	5.810e-12	-25.75
11	-25.87	-25.51	5.812e-12	-25.74
12	-25.87	-25.47	5.842e-12	-25.72

From figure 5, we can see different information criteria gives different results. One lag and nine lags are the two options from the results. We can run one set of a VAR model with one lag and one set of a VAR model with nine lags and then compare the model results and select the optimal one. In this case, we are going to select only one lag for the purpose of demonstration.

Now let's run the VAR(1) model for the difference of Google's stock price, EUR/USD Forex rate, and U.S. 10-year Treasury yield. We will include an intercept in this model.

Figure 6: VAR(1) Model Result for Differenced Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield

```
[8]: # VAR(1) model for Differenced GOOGLE, EURUSD, UST10Y
diff_mod = VAR(diff_data)
diff_mod_var = diff_mod.fit(
    maxlags=None,
    # when maxlags=None criterion to use for VAR order selection is
    # ic{'aic', 'fpe', 'hqic', 'bic', None}
    ic=None, # ic=None => automatic lag selection
    method="ols",
    trend="c",
    verbose=True,
)
diff_mod_var.summary()
```

```
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
    self._init_dates(dates, freq)
```

```
[8]: Summary of Regression Results
=====
Model:                VAR
Method:               OLS
Date:                Tue, 23, Dec, 2025
```


Time: 08:46:32

```
-----
No. of Equations:      3.00000    BIC:                -25.7532
Nobs:                  1503.00    HQIC:               -25.7799
Log likelihood:        12999.5    FPE:                6.26733e-12
AIC:                   -25.7957    Det(Omega_mle):     6.21756e-12
-----
```

Results for equation GOOGLE

```
=====
              coefficient      std. error      t-stat      prob
-----
const          0.000992        0.000420        2.362        0.018
L1.GOOGLE      -0.098637        0.026278       -3.754        0.000
L1.EURUSD       0.005368        0.094335        0.057        0.955
L1.UST10Y      -0.035157        0.012097       -2.906        0.004
=====
```

Results for equation EURUSD

```
=====
              coefficient      std. error      t-stat      prob
-----
const          0.000018        0.000114        0.159        0.874
L1.GOOGLE       0.012124        0.007159        1.694        0.090
L1.EURUSD      -0.020967        0.025700       -0.816        0.415
L1.UST10Y      -0.013055        0.003296       -3.961        0.000
=====
```

Results for equation UST10Y

```
=====
              coefficient      std. error      t-stat      prob
-----
const          -0.000106        0.000918       -0.115        0.908
L1.GOOGLE      -0.179094        0.057409       -3.120        0.002
L1.EURUSD       0.272183        0.206094        1.321        0.187
L1.UST10Y       0.001932        0.026429        0.073        0.942
=====
```

Correlation matrix of residuals

```
          GOOGLE    EURUSD    UST10Y
GOOGLE    1.000000    0.013730    0.220933
EURUSD     0.013730    1.000000   -0.016592
UST10Y     0.220933   -0.016592    1.000000
```

```
[9]: # Get the lag order that was selected
lag_order = diff_mod_var.k_ar
```

```
print(lag_order)
```

1

Figure 6 lists out the coefficient estimates for three equations. For the Google equation, Google lag 1 and 10-year treasury yield lag 1 are significant. For EUR/USD Forex rate, 10-year Treasury yield lag 1 is significant. For 10-year Treasury yield, Google lag 1 is significant. Now, let's check out the forecast plots from the model.

Figure 7: VAR Model Forecast of the Difference of the Differenced Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield

```
[10]: # VAR Model Forecast of the Difference of the Differenced GOOGLE, EURUSD, UST10Y
diff_mod_var.plot_forecast(steps=100, alpha=0.05, plot_stderr=True)
plt.show()
```

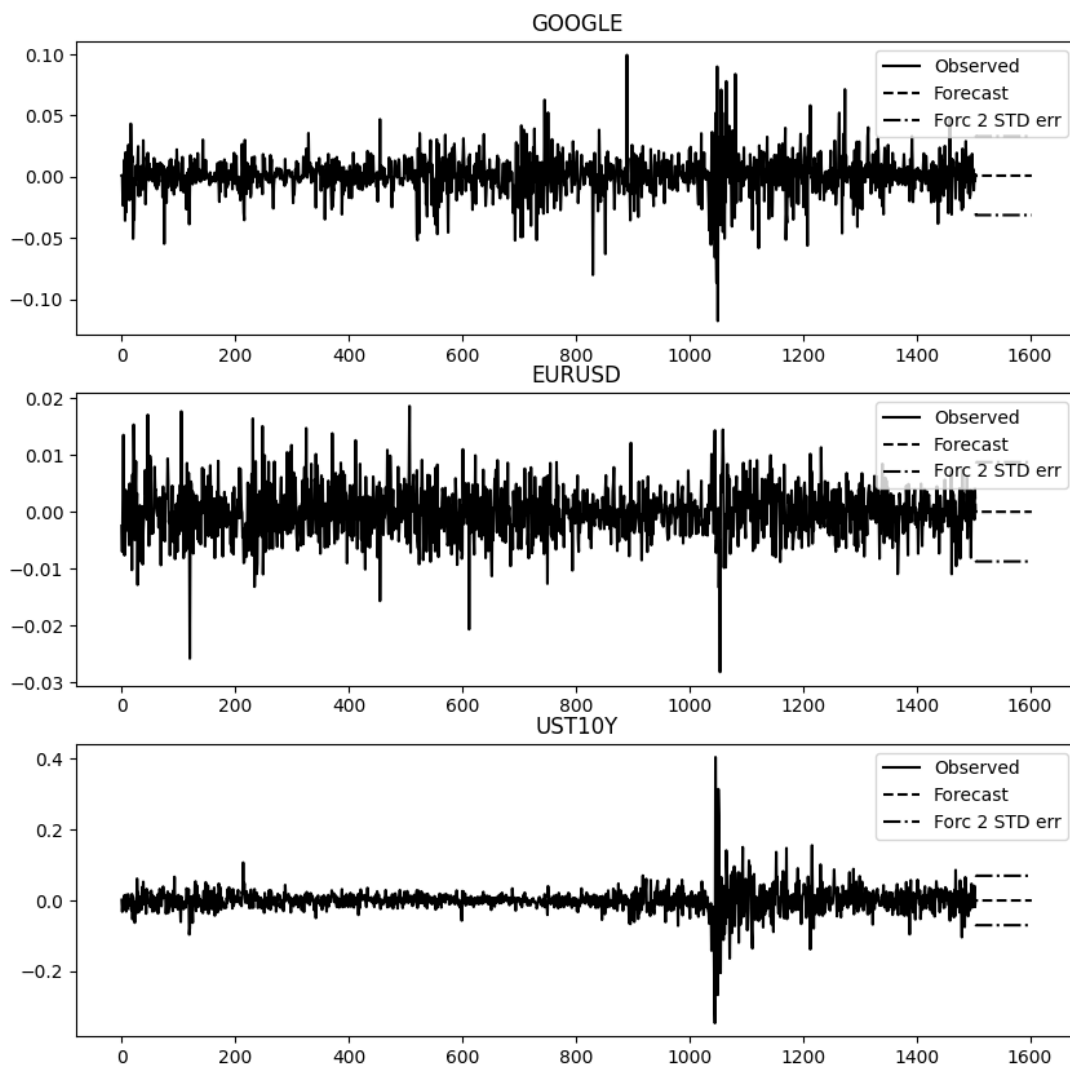


Figure 8: VAR Model Forecast of Level of Google Stock Price, EUR/USD Forex Rate, and U.S. 10-Year Treasury Yield

```
[11]: # VAR Model Forecast of Level of GOOGLE, EURUSD, UST10Y

# get 100 step prediction values for differenced data
# drop NA values at start
diff_data_val = diff_data.values[diff_mod_var.k_ar :] # noQA E203
pred = diff_mod_var.forecast(y=diff_data_val, steps=100)
pd.DataFrame(pred)

# extend index with 100 more business dates per USFederalHolidayCalendar
us_bd = CustomBusinessDay(calendar=USFederalHolidayCalendar())
idx = pd.date_range("2022-01-03", periods=100, freq=us_bd)
df_forecast = pd.DataFrame(
    data=pred, index=idx, columns=["GOOGLE1d", "EURUSD1d", "UST10Y1d"]
)

# Get the last log values
last_log_goog = np.log(data_set["GOOGLE"].iloc[-1])
last_log_eur = np.log(data_set["EURUSD"].iloc[-1])
last_log_ust10 = np.log(data_set["UST10Y"].iloc[-1])

# Convert log-differences to log-levels by cumulative sum
df_forecast["GOOGLE_log_level"] = last_log_goog + df_forecast["GOOGLE1d"].
    ↪cumsum()
df_forecast["EURUSD_log_level"] = last_log_eur + df_forecast["EURUSD1d"].
    ↪cumsum()
df_forecast["UST10Y_log_level"] = last_log_ust10 + df_forecast["UST10Y1d"].
    ↪cumsum()

# Convert log-levels back to original levels using exponential
df_forecast["GOOGLEForecast"] = np.exp(df_forecast["GOOGLE_log_level"])
df_forecast["EURUSDForecast"] = np.exp(df_forecast["EURUSD_log_level"])
df_forecast["UST10YForecast"] = np.exp(df_forecast["UST10Y_log_level"])

# plot initial data with forecasted values for each of GOOGLE, EURUSD, UST10Y
data_set["GOOGLE"].plot(figsize=(16, 5), legend=True)
df_forecast["GOOGLEForecast"].plot(legend=True)
plt.axvline(x="2022-01-03", color="b", alpha=0.5, linestyle="--")
plt.show()

data_set["EURUSD"].plot(figsize=(16, 5), legend=True)
df_forecast["EURUSDForecast"].plot(legend=True)
plt.axvline(x="2022-01-03", color="b", alpha=0.5, linestyle="--")
plt.show()
```

```
data_set["UST10Y"].plot(figsize=(16, 5), legend=True)
df_forecast["UST10YForecast"].plot(legend=True)
plt.axvline(x="2022-01-03", color="b", alpha=0.5, linestyle="--")
plt.show()
```



In figure 7, we can see the forecast plots for differenced time series. The forecast for the three time series stays flat. In figure 8, we converted the difference forecast to level forecast. Google

stock seems to stay flat in the level forecast. EUR/USD forecast is slightly up and the U.S. 10-year Treasury yield is slightly up.

1.4 4. Conclusion

In this lesson, we started by introducing the concept of ergodicity. Ergodicity is a time series property, which states that the values of a time series move around the mean of the process and as the number of data points increase, the more likely that these data points will cover all the possible values of the distribution of the process. We provided a counterexample to demonstrate the idea of ergodicity. We then gave an intuitive definition of ergodicity. We finished this topic by briefly discussing its importance for time series analysis.

We then moved on to the vector autoregressive (VAR) model. We went through the definition and properties of a VAR model. We concluded the lesson with a VAR model example for Google stock price, EUR/USD Forex rate, and U.S. 10-year Treasury yield. In the next lesson, we will continue our multivariate time series analysis.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Kempthorne, Peter et al. "Time Series Analysis III." Topics in Mathematics with Applications in Finance, Fall 2013, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-s096-topics-in-mathematics-with-applications-in-finance-fall-2013/2878a31dc5f908839d3c66f61f41f85c/MIT18_S096F13_lecnote12.pdf

- **Required pages:** Page 1-5
- **Estimated reading time:** 1 hour

Lesson 3 - Cointegration and Error Correction Model

December 23, 2025

0.1 FINANCIAL ECONOMETRICS

MODULE 6 | LESSON 3

1 COINTEGRATION AND ERROR CORRELATION MODEL

Reading Time	60 minutes
Prior Knowledge	Time series analysis, VAR model
Keywords	Spurious regression, Cointegration, Engle-Granger Two-Step Procedure, Error Correction Model

In this lesson, we are going to investigate the issue of using non-stationary time series to run a regression model. We will first point out the problem if both dependent time series and independent time series are non-stationary. Then, we will elaborate that, under certain circumstances, we can still use non-stationary time series to run a regression. That is when different time series are cointegrated in one model. Then, we will introduce a revised regression model for cointegrated time series. In this lesson, we will focus on only two time series. In the next lesson, we will extend the analysis to more than two time series.

```
[1]: # Load packages
import matplotlib.pyplot as plt
import pandas as pd
from arch.unitroot import ADF
#!pip install arch has previously been run
from arch.unitroot.cointegration import phillips_ouliaris
from statsmodels.stats.stattools import durbin_watson
from statsmodels.tsa.ardl import UECM

plt.rcParams["figure.figsize"] = (16, 9) # Figure size and width
```

```
[2]: # Download datasets
forex = pd.read_csv("M6. forex_1.csv")

# Convert date variable to date format
```

```
forex["Date2"] = pd.to_datetime(forex["Date"], format="%m/%d/%Y")
forex = forex.loc[:, ["Date2", "AUDUSD", "NZDUSD", "USDCAD"]].
↪set_index("Date2")[64:]
```

1.1 1. Spurious Regression

Spurious regression means one random walk time series runs an ordinary least squares regression on another random walk time series and the regression result will show a significant relationship even if the two random walk time series are independent. Granger and Newbold (1974) show that the regression result will likely show a non-existing relationship.

Assume we have time series x_t and y_t and both of them are $I(1)$. If we run the following ordinary least squares (OLS) regression:

$$y_t = \beta_0 + \beta_1 x_t + \epsilon_t$$

If ϵ_t is also $I(1)$, which means ϵ_t is serially correlated. We can write ϵ_t as follows:

$$\epsilon_t = \epsilon_{t-1} + \nu_t$$

where ν_t is stationary.

In this case, we cannot run this OLS regression because the error term cannot be serially correlated, and also, the variance will change over time. When the error term ϵ_t of OLS estimation is described like above, **Durbin-Watson statistic (DW test)** should be able to detect it. DW test is a test to check for autocorrelation for residuals of a model. The DW statistic has a value between 0 and 4. When the DW statistic is close to 2, there is no autocorrelation. To formally test for positive correlation or negative correlation, we need to use DW tables and identify two values d_L and d_U . Then

if $0 \leq DW < d_L$: positively correlated. if $d_U < DW \leq 4$: negatively correlated. if $d_L \leq DW \leq d_U$: test is inconclusive.

Since ϵ_t is $I(1)$, we can also write ϵ_t as follows:

$$\epsilon_t = \alpha \epsilon_{t-1} + \nu_t$$

where $\alpha = 1$.

From the above presentation, we can view ϵ_t as an AR process with unit root.

The common features from a spurious regression result are as follows:

- x_t and y_t will have a significant relationship even though the true value of $\beta_1 = 0$
- t-test for linearity will be significant
- R^2 will be high
- Low Durbin Watson Statistic

So how do we detect spurious regression problems? For the residuals from an OLS regression, we can draw ACF plot, run DW test and unit root test to check for the residuals to see if they are autocorrelated.

The lesson from spurious regression is to always check for stationarity of time series before using them in an OLS regression.

However, having x_t and y_t being $I(1)$ does not imply ϵ_t is always $I(1)$. When ϵ_t is $I(0)$ or stationary, we call x_t and y_t cointegrated.

1.2 2. Cointegration

1.2.1 2.1. The Basics

Time series x_t and y_t are cointegrated if they have the following properties:

- a. x_t and y_t are non-stationary.
- b. A stationary linear combination of x_t and y_t exists.

We can rephrase the above definition as follows: Assume $x_t \sim I(1)$ and $y_t \sim I(1)$. If we run an OLS regression of y on x or an OLS regression of x on y and the residual is stationary or $I(0)$, then x_t and y_t have the same stochastic trend and are cointegrated.

If x_t and y_t are cointegrated, then x_t and y_t have a long-term stable relationship even though they are non-stationary. We say that x_t and y_t have long-term equilibrium. It means in the short-term, x_t and y_t might deviate from each other, but they will ultimately revert back to their long-term relationship. We call it the **mean reversion property**. This property happens to many financial variables. One common example is the difference between long-term interest rates and short-term interest rates. In general, long-term interest rates and short-term interest rates are not stationary. However, their difference can be stationary. Let's look at one example between the Australian Dollar/U.S. Dollar (AUDUSD) and New Zealand Dollar/U.S. Dollar (NZDUSD) foreign exchange (forex) rates time series.

Figure 1: AUDUSD and NZDUSD Plot from 2016 to 2021

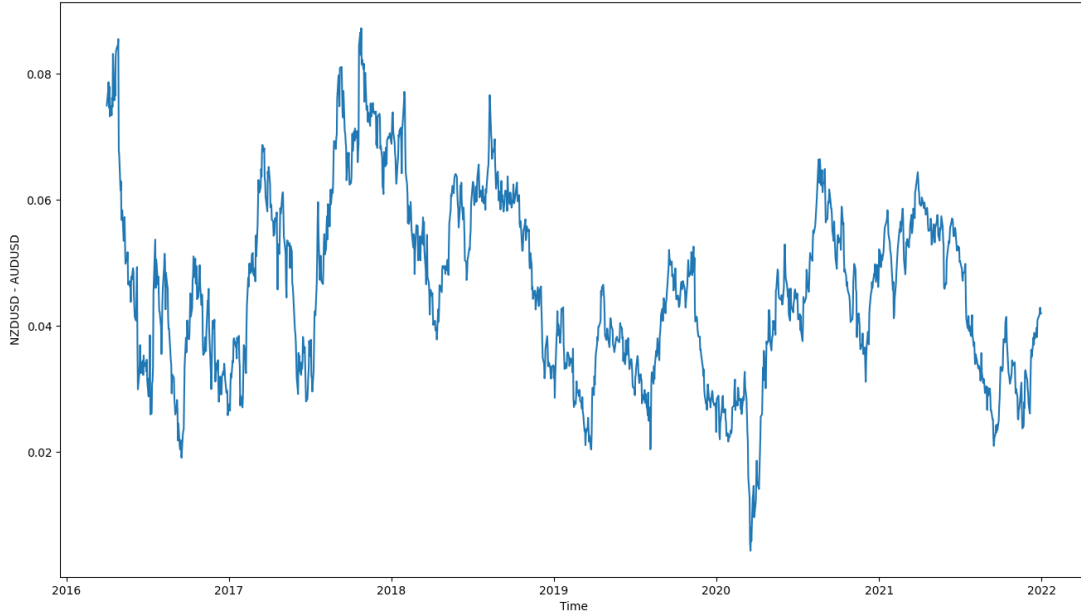
```
[3]: # AUDUSD and NZDUSD Plot from 2016 to 2021
plt.plot(forex.AUDUSD, linewidth=1, c="g", label="AUDUSD")
plt.plot(forex.NZDUSD, linewidth=1, c="r", label="NZDUSD")
plt.xlabel("Time")
plt.ylabel("NZDUSD & AUDUSD Forex Rates")
plt.legend()
plt.show()
```



Figure 1 shows the time plot for both AUDUSD and NZDUSD. From the plot, we can see the two forex rates move in tandem from 2016 to 2021. In figure 2, we show the difference between AUDUSD and NZDUSD. From the plot, the difference seems to be moving in a broad channel.

Figure 2: Difference of AUDUSD and NZDUSD Plot from 2016 to 2021

```
[4]: # Difference of AUDUSD and NZDUSD Plot from 2016 to 2021
plt.plot(forex.AUDUSD - forex.NZDUSD)
plt.xlabel("Time")
plt.ylabel("NZDUSD - AUDUSD")
plt.show()
```



1.2.2 2.2 Engle-Granger Two-Step Procedure (EG Test) to Test for Cointegration

In the last section, we talked about the definition of two time series cointegrating with each other. In this section, we are going to introduce methods to test for two time series to see if they are cointegrated. First, we will introduce the **Engle-Granger two-step procedure**. We'll call it the EG test for short in this lesson. This is a popular method to test for cointegration. In the next lesson, we will introduce another method.

Engle and Granger (1987) proposed a method to test for cointegration of two time series. The EG test is based on the definition of cointegration we discussed in the previous section.

The idea of EG test is as follows. Assume x_t and y_t are two $I(1)$ time series. We first run the following OLS regression:

$$y_t = \beta_0 + \beta_1 x_t + u_t$$

where u_t is the residuals and β_1 is the cointegration coefficient.

Sometimes, we call u_t an equilibrium error since it measures the deviation of the actions of x_t and y_t from long-term equilibrium. Then, we run an augmented Dickey-Fuller test on u_t to see if it has a unit root. If u_t has a unit root, it means that x_t and y_t are not cointegrated. If u_t has no unit root, then x_t and y_t are cointegrated. However, there is a twist. Since u_t is a residual from an OLS regression of two time series, we cannot use the regular Dickey-Fuller table to decide the critical values for testing. Phillips and Ouliaris (1990) (**PO Test**) provided a method and tables for critical values to check for unit roots for residuals. Instead of the ADF test, Phillips and Ouliaris use another popular unit root test, the Phillips-Perron Test, to check for unit root and generate the tables. In general, these types of tests we are using are **residual-based tests for cointegration**.

since we check residuals for unit root to decide cointegration. We also call it a **single equation approach** as we only run one OLS regression. The H_0 and H_1 of the test is as follows:

H_0 : u_t has unit root (non-stationary) / x_t and y_t are not cointegrated H_1 : u_t does not have unit root (stationary) / x_t and y_t are cointegrated

If we reject H_0 and confirm x_t and y_t are cointegrated, we say the estimate of β_1 from the OLS regression is **super-consistent**. Super-consistent means the sample estimate converges to the true population parameter. We do not need to take the difference of x_t and y_t before running an OLS regression. We do not have to worry about the spurious regression issue when x_t and y_t are cointegrated.

Let's summarize the steps to run an EG test.

- Check if x_t and y_t are $I(1)$.
- Run an OLS regression of x_t on y_t or y_t on x_t and obtain the residual u_t .
- Test u_t for a unit root by using Phillips and Ouliaris method or tables. If u_t has a unit root, then x_t and y_t are not cointegrated. If u_t does not have a unit root, then x_t and y_t are cointegrated.

1.2.3 2.3 EG Test for Cointegration Example: AUDUSD vs NZDUSD

After describing how to use the EG test to check for cointegration of two time series, let's apply it to AUDUSD and NZDUSD historical forex rates.

We first have to check if AUDUSD and NZDUSD are $I(1)$. In order to be $I(1)$, both time series should have a unit root for the ADF test for the levels of AUDUSD and NZDUSD.

Figure 3: ADF Test Results for AUDUSD and NZDUSD Forex Rates From 2016 to 2021 All rights reserved WQU WorldQuant University QQQQ

```
[5]: # ADF Test Results for AUDUSD
AUDUSD_adf = ADF(forex.AUDUSD, trend="n", method="bic")
print(
    "AUDUSD Augmented Dickey-Fuller Unit Root Test\n", AUDUSD_adf.regression.
    ↪summary()
)
print("\nTest statistics and critical values: \n", AUDUSD_adf)

# ADF Test Results for NZDUSD
print("\n", "# " * 39, "\n")
NZDUSD_adf = ADF(forex.NZDUSD, trend="n", method="bic")
print(
    "NZDUSD Augmented Dickey-Fuller Unit Root Test\n", NZDUSD_adf.regression.
    ↪summary()
)
print("\nTest statistics and critical values: \n", NZDUSD_adf)
```

AUDUSD Augmented Dickey-Fuller Unit Root Test

OLS Regression Results

=====

```

=====
Dep. Variable:          y    R-squared (uncentered):
0.000
Model:                OLS    Adj. R-squared (uncentered):
-0.001
Method:              Least Squares    F-statistic:
0.1392
Date:                Tue, 23 Dec 2025    Prob (F-statistic):
0.709
Time:                08:49:04    Log-Likelihood:
6067.5
No. Observations:      1497    AIC:
-1.213e+04
Df Residuals:          1496    BIC:
-1.213e+04
Df Model:              1
Covariance Type:      nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Level.L1	-5.522e-05	0.000	-0.373	0.709	-0.000	0.000

```

=====
Omnibus:              50.793    Durbin-Watson:              1.996
Prob(Omnibus):         0.000    Jarque-Bera (JB):          118.577
Skew:                  -0.147    Prob(JB):                  1.78e-26
Kurtosis:              4.347    Cond. No.                  1.00
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:

Augmented Dickey-Fuller Results

```

=====
Test Statistic        -0.373
P-value               0.547
Lags                  0
=====

```

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

#####

NZDUSD Augmented Dickey-Fuller Unit Root Test

OLS Regression Results

=====						
=====						
Dep. Variable:	y	R-squared (uncentered):				
0.000						
Model:	OLS	Adj. R-squared (uncentered):				
-0.001						
Method:	Least Squares	F-statistic:				
0.02979						
Date:	Tue, 23 Dec 2025	Prob (F-statistic):				
0.863						
Time:	08:49:04	Log-Likelihood:				
6094.7						
No. Observations:	1497	AIC:				
-1.219e+04						
Df Residuals:	1496	BIC:				
-1.218e+04						
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Level.L1	-2.676e-05	0.000	-0.173	0.863	-0.000	0.000
=====						
Omnibus:	30.091	Durbin-Watson:		2.011		
Prob(Omnibus):	0.000	Jarque-Bera (JB):		55.722		
Skew:	-0.105	Prob(JB):		7.95e-13		
Kurtosis:	3.922	Cond. No.		1.00		
=====						

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:

Augmented Dickey-Fuller Results

=====	
Test Statistic	-0.173
P-value	0.624
Lags	0

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

Figure 3 shows the result of the ADF tests on levels of AUDUSD and NZDUSD. We can see that we cannot reject H_0 that there is a unit root for both cases. Now let's check the difference of AUDUSD and NZDUSD.

Figure 4: ADF Test Results for First Difference of AUDUSD and NZDUSD Forex Rates from 2016 to 2021

```
[6]: # ADF Test Results for First Difference of AUDUSD Forex Rates
AUDUSD_d_adf = ADF(forex.AUDUSD.diff().dropna(), trend="n", method="bic")
print(
    "First Difference of AUDUSD Augmented Dickey-Fuller Unit Root Test\n",
    AUDUSD_d_adf.regression.summary(),
)
print("\nTest statistics and critical values: \n", AUDUSD_d_adf)

# ADF Test Results for First Difference of NZDUSD Forex Rates
print("\n", "# " * 39, "\n")
NZDUSD_d_adf = ADF(forex.NZDUSD.diff().dropna(), trend="n", method="bic")
print(
    "First Difference of NZDUSD Augmented Dickey-Fuller Unit Root Test\n",
    NZDUSD_d_adf.regression.summary(),
)
print("\nTest statistics and critical values: \n", NZDUSD_d_adf)
```

First Difference of AUDUSD Augmented Dickey-Fuller Unit Root Test

OLS Regression Results

```
=====
=====
Dep. Variable:          y      R-squared (uncentered):
0.499
Model:                OLS      Adj. R-squared (uncentered):
0.499
Method:                Least Squares      F-statistic:
1489.
Date:                  Tue, 23 Dec 2025      Prob (F-statistic):
1.15e-226
Time:                  08:49:05      Log-Likelihood:
6062.8
No. Observations:      1496      AIC:
-1.212e+04
Df Residuals:          1495      BIC:
-1.212e+04
Df Model:              1
Covariance Type:       nonrobust

=====
=====
```

	coef	std err	t	P> t	[0.025	0.975]
--	------	---------	---	------	--------	--------

```

-----
Level.L1      -0.9981      0.026      -38.592      0.000      -1.049      -0.947
=====
Omnibus:                50.561      Durbin-Watson:                1.998
Prob(Omnibus):           0.000      Jarque-Bera (JB):            117.849
Skew:                    -0.147      Prob(JB):                    2.57e-26
Kurtosis:                4.343      Cond. No.                    1.00
=====

```

Notes:

- [1] R^2 is computed without centering (uncentered) since the model does not contain a constant.
- [2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:

Augmented Dickey-Fuller Results

```

=====
Test Statistic           -38.592
P-value                   0.000
Lags                      0
-----

```

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

#####

First Difference of NZDUSD Augmented Dickey-Fuller Unit Root Test

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared (uncentered):
0.503
Model:                  OLS    Adj. R-squared (uncentered):
0.503
Method:                  Least Squares    F-statistic:
1512.
Date:                    Tue, 23 Dec 2025    Prob (F-statistic):
3.87e-229
Time:                    08:49:05    Log-Likelihood:
6090.3
No. Observations:        1496    AIC:
-1.218e+04
Df Residuals:            1495    BIC:
-1.217e+04

```



```

Df Model:                                1
Covariance Type:                        nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Level.L1      -1.0056      0.026     -38.886      0.000      -1.056      -0.955
=====
Omnibus:                        30.377    Durbin-Watson:                        1.999
Prob(Omnibus):                    0.000    Jarque-Bera (JB):                        56.319
Skew:                          -0.107    Prob(JB):                        5.90e-13
Kurtosis:                       3.926    Cond. No.                        1.00
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:

Augmented Dickey-Fuller Results

```

=====
Test Statistic      -38.886
P-value              0.000
Lags                 0
-----

```

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

Figure 4 shows the ADF test results for the first difference of AUDUSD and NZDUSD. In both cases, we can easily reject H_0 that a unit root exists. Hence, we can claim that both AUDUSD and NZDUSD are $I(1)$.

The next step is to run an OLS model and then test the residuals for a unit root. As described in the last section, we will use Phillips and Ouliaris method and the table (PO test) to check for stationarity of residuals. We present the results in the following figure.

Figure 5: Two-Step Residual Based Test for Cointegration for AUDUSD and NZDUSD

```

[7]: # Two-Step Residual Based Test for Cointegration for AUDUSD and NZDUSD
      # (aka Phillips and Ouliaris cointegration test)
      print(
        phillips_ouliaris(
          forex.AUDUSD, forex.NZDUSD, trend="c", test_type="Za", kernel="bartlett"
        )
      )

```

Phillips-Ouliaris Za Cointegration Test
 Statistic: -28.879606159224416
 P-value: 0.0083708705680421
 Null: No Cointegration, Alternative: Cointegration
 Kernel: Bartlett
 Bandwidth: 6.4927
 Trend: c
 Distribution Order: 3

From figure 5, we can see the p -value is less than 0.05 critical value. Hence, we can reject H_0 and claim that AUDUSD and NZDUSD are cointegrated.

1.3 3. Error Correction Model

In section 1, we talked about the fact that if we run a regression on non-stationary time series, we might run into a spurious regression issue. In section 2, we went through the situation when two time series are cointegrated, the residuals of their regression will be stationary. We also discussed the fact that when time series are cointegrated, there exists a long-term equilibrium among them. We have already seen many different financial data moving in different directions in short term but in the long run they display a consistent relationship. How can we use what we have learned about cointegration to build a financial model and analyze the behavior of cointegrated non-stationary time series? The error correction model provides an answer for us.

Error correction model (ECM) is actually a continuation of cointegration analysis. Let's use an example to understand what ECM is. Assume we have two cointegrated non-stationary time series x_t and y_t .

Since x_t and y_t are cointegrated, they exist in a long-term relationship as follows:

$$y_t = \beta_0 + \beta_1 x_t + \epsilon_t$$

We can rewrite the relationship as follows:

$$\epsilon_t = y_t - \beta_0 - \beta_1 x_t$$

However, we would like to build a model that can include both long-term relationships and short-term fluctuations. We still want y_t to be the dependent time series and x_t to be an independent time series. We also will include lag terms x_{t-1} and y_{t-1} so the model can account for the impact of time series in the previous time points. Now, we can write a new regression model as follows:

$$y_t = \theta_0 + \theta_1 x_t + \theta_2 x_{t-1} + \phi y_{t-1} + u_t$$

Where u_t is serially uncorrelated white noise.

With some manipulation and substitutions from the long-term equilibrium (cointegration) equation, we can rewrite the above regression model as follows: (proof is in the ECM Proof section)

$$\Delta y_t = \alpha_0 + \alpha_1 \Delta x_t + \lambda[y_{t-1} - \beta_0 - \beta_1 x_{t-1}] + u_t$$

Where

$$\alpha_0 = \theta_0 - (1 - \phi)\beta_0 \quad \alpha_1 = \theta_1 \quad \lambda = -(1 - \phi) \quad \beta_1 = \frac{\theta_1 + \theta_2}{1 - \phi}$$

If we replace the formula inside the square brackets in the equation above with the long-term equilibrium equation, we get:

$$\Delta y_t = \alpha_0 + \alpha_1 \Delta x_t + \lambda[\epsilon_{t-1}] + u_t$$

This is the basic setup for an error correction model. This is a very powerful model that combines the short-term behavior and long-term behavior of x_t and y_t .

a) Δx_t : This part captures the short-term impact of the movement of x_t on y_t .

b) We know both x_t and y_t are $I(1)$, so Δx_t and Δy_t are $I(0)$ and stationary. Also, since x_t and y_t are cointegrated, the residuals ϵ_t from their regression will be $I(0)$ too. Therefore, in ECM regression, all our terms are stationary. We will not have the spurious regression issue.

c) $\lambda[\epsilon_{t-1}]$: This is the error correction term in the regression. This is the part of the regression that captures the long-term dynamics of x_t and y_t . What does it mean? Say at time $t - 1$, ϵ_{t-1} is positive. It means $y_{t-1} > \beta_0 + \beta_1 x_{t-1}$. Hence, there is a disequilibrium happening for x and y at $t-1$. Assume λ is negative. At time t , according to the regression model, λ is negative, ϵ_{t-1} is positive, and Δy_t will be negative at time t . It means that at time t , y will decrease in order to return to a long-term equilibrium state. This is why this term is called the error correction term and why this model is called error correction model. This term provides the ability to model the force to bring short-term data deviation back to a long-term equilibrium state. λ is called the error correction coefficient. It adjusts the speed at which the data will go back to its long-term equilibrium state. In order to have the error correction function work, we need to make sure that λ is negative. Some literature will say λ needs to be positive because they separate the negative sign out from λ . Be careful when you read other ECM materials. We usually require λ to be between 0 and -1 since in most cases the full adjustment does not happen right away.

Before we conclude the theory of this lesson, let's recap what we have learned.

a. When all time series are stationary -> Run OLS regression or VAR model

b. When two time series are not stationary: - If non-stationary time series are not cointegrated -> Take the first difference of non-stationary time series and check for stationarity before running an OLS regression. - If non-stationary time series are cointegrated -> Error correction model

c. When one time series is not stationary and the other one is stationary -> Take the first difference of the non-stationary time series and check for stationarity before running an OLS regression.

d. When there are more than two non-stationary time series -> We will discuss this situation in the next lesson.

1.4 4. ECM Model Example: AUDUSD vs NZDUSD

In this section, let's run an ECM model on AUDUSD and NZDUSD. AUDUSD will be the dependent variable and NZDUSD will be the independent variable. In the last section, we have already checked that both AUDUSD and NZDUSD are $I(1)$ and they are cointegrated. Hence, we can run an ECM on them. Figure 6 shows the model results.

Figure 6: Error Correction Model Result for AUDUSD on NZDUSD from 2016 to 2021

[8]: *# Error Correction Model Result for AUDUSD on NZDUSD*

```
ecm_model = UECM(
    endog=pd.DataFrame(forex.AUDUSD),
    lags=1,
    exog=pd.DataFrame(forex.NZDUSD),
    order=1,
    trend="c",
).fit()
print(ecm_model.summary())
```

UECM Model Results						
=====						
Dep. Variable:	D.AUDUSD		No. Observations:		1498	
Model:	UECM(1, 1)		Log Likelihood		6872.820	
Method:	Conditional MLE		S.D. of innovations		0.734	
Date:	Tue, 23 Dec 2025		AIC		-13735.640	
Time:	08:49:06		BIC		-13709.084	
Sample:	1		HQIC		-13725.746	
	1498					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	0.0001	0.001	0.099	0.921	-0.002	0.003
AUDUSD.L1	-0.0157	0.005	-3.469	0.001	-0.025	-0.007
NZDUSD.L1	0.0165	0.005	3.093	0.002	0.006	0.027
D.NZDUSD.L0	0.8275	0.015	53.588	0.000	0.797	0.858
=====						

```
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
    self._init_dates(dates, freq)
```

Figure 6 shows the ECM results for AUDUSD and NZDUSD. From the results, we can see all coefficient estimates are significant. λ is AUDUSD.L1 = -0.0157, which meets our criteria to be between 0 and -1. It means if the AUDUSD exchange rate deviates 1 point from the long-term equilibrium with NZDUSD at time $t-1$, AUDUSD will drop 0.0157 point at time t to bring it back to long-term equilibrium.

Next, let's check the residuals to see if there is any autocorrelation issue. We will run the Durbin-Watson test. Figure 7 shows the result.

Figure 7: Durbin-Watson Test Result for ECM of AUDUSD and NZDUSD

```
[9]: # Durbin-Watson Test Result for the residuals of ECM of AUDUSD and NZDUSD
residuals = ecm_model.resid

# Durbin-Watson test
dw_stat = durbin_watson(residuals)
print(f"Durbin-Watson statistic: {dw_stat:.5f}")
```

Durbin-Watson statistic: 0.00003

From figure 7, we can see the residuals does not pass Durbin-Watson test. It means that the residuals are still serially correlated.

1.5 5. ECM Proof

Assume we have two cointegrated $I(1)$ time series x_t and y_t . We can write the long-term equilibrium as follows:

$$y_t = \beta_0 + \beta_1 x_t + \epsilon_t$$

We would like to build a regression model for x_t and y_t that can account for their long-term relationship and short-term dynamics. We can write the model as the following regression:

$$y_t = \theta_0 + \theta_1 x_t + \theta_2 x_{t-1} + \phi y_{t-1} + u_t$$

Where u_t is serially uncorrelated white noise

First, we add lag term y_{t-1} to both side of the regression.

$$y_t - y_{t-1} = \theta_0 + \theta_1 x_t + \theta_2 x_{t-1} + \phi y_{t-1} - y_{t-1} + u_t$$

$$\Rightarrow \Delta y_t = \theta_0 + \theta_1 x_t + \theta_2 x_{t-1} - (1 - \phi) y_{t-1} + u_t$$

We then add these two terms into the right-hand side of the regression $-\theta_1 x_{t-1}$ and $\theta_1 x_{t-1}$.

$$\Delta y_t = \theta_0 + \theta_1 x_t - \theta_1 x_{t-1} + \theta_1 x_{t-1} + \theta_2 x_{t-1} - (1 - \phi) y_{t-1} + u_t$$

$$\Rightarrow \Delta y_t = \theta_0 + \theta_1 \Delta x_t + (\theta_1 + \theta_2) x_{t-1} - (1 - \phi) y_{t-1} + u_t$$

$$\Rightarrow \Delta y_t = \theta_0 + \theta_1 \Delta x_t - (1 - \phi) \left[y_{t-1} - \frac{\theta_1 + \theta_2}{1 - \phi} x_{t-1} \right] + u_t$$

Then, we add these two terms into the right-hand side of the regression $-(1 - \phi)\beta_0$ and $(1 - \phi)\beta_0$.

$$\Delta y_t = \theta_0 - (1 - \phi)\beta_0 + \theta_1 \Delta x_t - (1 - \phi)[y_{t-1} - \beta_0 - \frac{\theta_1 + \theta_2}{1 - \phi} x_{t-1}] + u_t$$

We can rewrite the above regression as follows:

$$\Delta y_t = \alpha_0 + \alpha_1 \Delta x_t + \lambda[y_{t-1} - \beta_0 - \beta_1 x_{t-1}] + u_t$$

Where

$$\alpha_0 = \theta_0 - (1 - \phi)\beta_0 \quad \alpha_1 = \theta_1 \quad \lambda = -(1 - \phi) \quad \beta_1 = \frac{\theta_1 + \theta_2}{1 - \phi}$$

We can also replace the formula in the [] with the error term in $t-1$ from the long-term equilibrium equation.

$$\Delta y_t = \alpha_0 + \alpha_1 \Delta x_t + \lambda[\epsilon_{t-1}] + u_t$$

1.6 6. Conclusion

In this lesson, we looked into when we may have to run a regression on non-stationary time series. We discussed the issue of spurious regression. We then introduced the concept of cointegration among two non-stationary time series. If the two time series are cointegrated, we can run an OLS regression without issue. Lastly, we introduced the error correction model, which can model both the short-term activity and long-term equilibrium of two cointegrated time series. In next lesson, we will extend our analysis to multiple time series.

1.7 References

- Engle, Robert F., and C. W. J. Granger. “Cointegration and Error Correction: Representation, Estimation and Testing.” *Econometrica*, vol. 55, no. 2, 1987, pp. 251–276.
- Granger, C. W. J., and P. Newbold. “Spurious Regressions in Econometrics.” *Journal of Econometrics*, vol. 2, no. 2, 1974, pp. 111–120.
- Phillips, P. C. B., and S. Ouliaris. “Asymptotic Properties of Residual Based Tests for Cointegration.” *Econometrica*, vol. 58, no. 1, 1990, pp. 165–193.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Kempthorne, Peter et al. "Time Series Analysis III." Topics in Mathematics with Applications in Finance, Fall 2013, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-s096-topics-in-mathematics-with-applications-in-finance-fall-2013/2878a31dc5f908839d3c66f61f41f85c/MIT18_S096F13_lecnote12.pdf

- **Required pages:** Page 6-12
- **Estimated reading time:** 1 hour

FINANCIAL ECONOMETRICS

MODULE 6 | LESSON 4

VECTOR ERROR CORRECTION MODEL AND JOHANSEN TEST FOR COINTEGRATION

Reading Time 60 minutes

Prior Knowledge Cointegration for two time series, Error Correction Model

Keywords Integrated Time Series, Vector Error Correction Model (VECM), Johansen Test, Trace Test, Maximum Eigenvalue Test

In last lesson, we discussed the cointegration test for two time series. We also introduced Error Correction Model (ECM). In this lesson, we will look into a cointegration test for more than two time series. First, we will review the cointegration concept with more than two time series. Then, we will introduce the vector error correction model (VECM) as this is the model structure used for multivariate cointegration analysis. Next, we will introduce Johansen cointegration test for more than two time series.

```
In [1]: # Load packages
import matplotlib.pyplot as plt
import pandas as pd
from arch.unitroot import ADF
from statsmodels.tsa.api import VAR
from statsmodels.tsa.vector_ar.vecm import VECM, coint_johansen

plt.rcParams["figure.figsize"] = (16, 9) # Figure size and width
```

```
In [2]: # Download datasets
forex = pd.read_csv("./M6. forex_1.csv")

# Convert date variable to date format
forex["Date2"] = pd.to_datetime(forex["Date"], format="%m/%d/%Y")
forex = forex.loc[:, ["Date2", "AUDUSD", "NZDUSD", "USDCAD"]].set_index("Date2")[64
```

1. Cointegration with More Than Two Time Series

First, let's review the concept of cointegration in the case of multiple time series. An integrated time series of order d can be written as $I(d)$. When we take the difference d times, this series will turn into a stationary, $I(0)$, series. In this lesson, we will only deal with integrated time series of $I(1)$.

Assume we have three $I(1)$ time series, x_t , y_t and z_t . If there exists a linear combination:

$$\beta_1 x_t + \beta_2 y_t + \beta_3 z_t \equiv W_t$$

that is stationary or $I(0)$, where β is a 3×1 vector with β_1 , β_2 and β_3 as its elements and W_t is a 1×1 vector with x_t , y_t and z_t as elements.

Then, we say that x_t , y_t and z_t are cointegrated.

2. Vector Error Correction Model (VECM)

Vector error correlation models are related to vector autoregressive models (VARs) introduced in Module 6, Lesson 2. If we have three time series, x_t , y_t and z_t , we can write the VAR(1) model with these three time series as follows:

$$\begin{aligned} x_t &= \alpha_0 + \alpha_1 x_{t-1} + \alpha_2 y_{t-1} + \alpha_3 z_{t-1} + u_t \\ y_t &= \beta_0 + \beta_1 x_{t-1} + \beta_2 y_{t-1} + \beta_3 z_{t-1} + v_t \\ z_t &= \gamma_0 + \gamma_1 x_{t-1} + \gamma_2 y_{t-1} + \gamma_3 z_{t-1} + w_t \end{aligned}$$

Where u_t , v_t and w_t are white noise with 0 mean and constant variance.

Let's rewrite the VAR(1) model above in matrix form as follows:

$$X_t = C_0 + \Pi_0 X_{t-1} + \epsilon_t$$

Where C_0 is a 3×1 constant vector:

$$C_0 = \begin{bmatrix} \alpha_0 \\ \beta_0 \\ \gamma_0 \end{bmatrix}$$

X_t is a 3×1 vector:

$$X_t = \begin{bmatrix} x_t \\ y_t \\ z_t \end{bmatrix}$$

Π_0 is a 3×3 matrix:

$$\Pi_0 = \begin{bmatrix} \alpha_1 & \alpha_2 & \alpha_3 \\ \beta_1 & \beta_2 & \beta_3 \\ \gamma_1 & \gamma_2 & \gamma_3 \end{bmatrix}$$

ϵ_t is a 3×1 white noise vector:

$$\begin{bmatrix} u_t \\ v_t \\ w_t \end{bmatrix}$$

We know if x_t , y_t and z_t are all stationary, we can use the above VAR(1) model.

On the other hand, when they are not stationary, but their first differences are stationary, then we can use their first differences to model them as a VAR(1). The matrix form of the model can be written as follows:

$$\Delta X_t = C_1 + \Pi_1 \Delta X_{t-1} + \epsilon_t$$

If x_t , y_t and z_t are not stationary but they are cointegrated, then we can adjust the above model with an error correction term. This adjusted model is called a vector error correction model (VECM). We can write the matrix form of a VEC model as follows:

$$\Delta X_t = C + \Pi X_{t-1} + \Gamma \Delta X_{t-1} + \epsilon_t$$

In general, we need more than one lag for the difference term on the right-hand side of the equation system. We will discuss later how to find the right number of lags in the system. For now, let's assume the number of lags to be p . Hence, we can write our general VEC model as follows:

$$\Delta X_t = C + \Pi X_{t-1} + \sum_{i=1}^{p-1} \Gamma_i \Delta X_{t-i} + \epsilon_t$$

Where

- C is the deterministic vector or matrix depending if there are constants and/or linear trend
- Π is a 3×3 matrix for coefficients of lagged X_{t-1} vector
- Γ_i is a 3×3 matrix for coefficients of lagged differences of X
- ϵ_t is a white noise 3×1 vector with 0 means and stable covariance

The above VEC model was introduced by Johansen. The major difference between a VEC model and VAR model is the new term ΠX_{t-1} in the VEC model. ΠX_{t-1} is the error correction term for a VEC model. This error correction term in a VEC model is the long-term equilibrium part of the model. $\sum_{i=1}^{p-1} \Gamma_i \Delta X_{t-i}$ is the short-term movement part of the VEC model. Hence, VEC model can be used to analyze both long-term and short-term dynamics of cointegrated nonstationary $I(1)$ time series variables. We still assume x_t , y_t and z_t are $I(1)$.

Let's focus on the coefficient matrix Π for now. We can interpret Π as following three scenarios:

a. The rank of Π is 0

If $\Pi = 0$, that is all elements of Π are 0 , or the rank of Π is 0 , then there are no cointegrations among x_t , y_t and z_t . If all time series are $I(1)$, the VECM will reduce to a VAR(p) model with the first differences of all time series.

b. The rank of Π is full rank K

If Π has full rank K , (the diagonal elements of the matrix are all non-zero, in our case, $K=3$, because we have 3 time series), then x_t , y_t and z_t are stationary. There are no unit roots from all time series. We can apply the original levels of these time series to a VAR(p) model with no need to use a VEC model. This situation should not happen as we usually assume x_t , y_t , and z_t are $I(1)$.

c. The rank of Π is between 0 and K

If the rank of Π is m and $0 < m < K$, it means Π has reduced rank. This is the case with cointegration. Π can be split into two terms as follows:

$$\Pi = \alpha \beta'$$

Where Π is a $K \times K$ matrix, α is a $K \times m$ matrix, and β is a $K \times m$ matrix.

The columns of β matrix contain the m co-integrating vectors, and the columns of α contain the m adjustment vectors. $\text{Rank}(\Pi) = \text{Rank}(\alpha) = \text{Rank}(\beta) = m$. Co-integrating vectors include coefficients of a linear combination of co-integrating time series. There are m linear combinations from all time series that are stationary. Hence, there are $(k-m)$ unit roots from all time series.

Adjustment vectors contain the coefficients that can bring the short-term deviation of the time series back to their long-term equilibrium relationship. The coefficients in adjustment vectors are error correction coefficients. Therefore, Π corrects the error gap that happened in the last period and brings short-term disequilibrium among time series back to their long-term equilibrium.

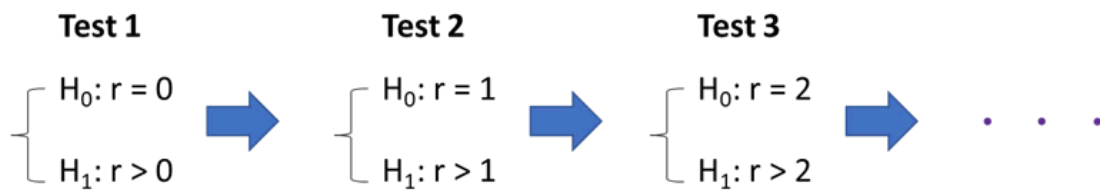
Now we know that when we have multiple non-stationary co-integrating time series, we should use a VEC model. However, how do we test multiple non-stationary time series for cointegration? This will be the topic of our next section.

3. Johansen Cointegration Test

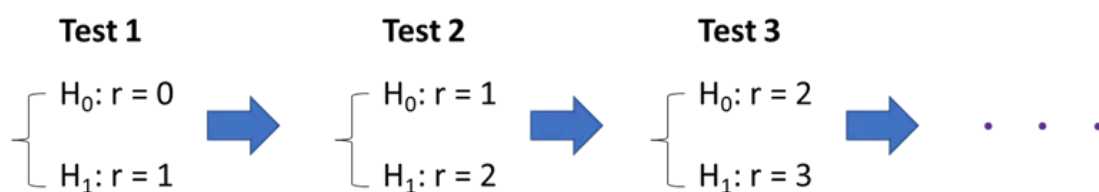
There are two types of sequential tests proposed by Johansen to test cointegration for multiple time series: trace test and maximum eigenvalue test. They are both likelihood-ratio tests. Both tests run a series of tests on the rank of Π we discussed in the last section to decide the number of cointegration relationships in a time series vector. These two tests have the same null hypothesis. The difference is the alternative hypothesis. Let's use the following example to explain how the two tests work.

Figure 1: Johansen Tests: Trace Test and Eigenvalue Test Demonstration

Johansen Cointegration Test: Trace Test



Johansen Cointegration Test: Maximum Eigenvalue Test



In figure 1, we show the hypothesis test setup for trace test and maximum eigenvalue test. r is the rank of Π . We can see both null hypotheses start from rank = 0, i.e., there is no cointegration. If we reject test 1, we move on to test 2. If we reject test 2, we move on to test 3 until we cannot reject null hypothesis or until we reach $K-1$ rank. K is the total number of time series in the time series vector for testing. Once we stop, we can decide what the rank is and how many linear combinations or co-integrating relationships exist.

In the next section, we will use three foreign exchange rates to demonstrate Johansen tests to identify a number of co-integrating relationships in the three foreign exchange rates.

4. Johansen Test Application: AUDUSD, NZDUSD, and CADUSD

In this section, we will use what we learned from last section to find the number of co-integrating relationships among AUDUSD, NZDUSD, and CADUSD foreign exchange rates from 2016 to 2021. In this example, we will use software to conduct the test. Let's draw the time plot of the three time series first.

Figure 2: AUDUSD, NZDUSD, and CADUSD Foreign Exchange Rates Time Plot from 2016 to 2021

```
In [3]: # AUDUSD, NZDUSD, and CADUSD Foreign Exchange Rates Time Plot
plt.plot(forex.AUDUSD, linewidth=1, c="g", label="AUDUSD")
plt.plot(forex.NZDUSD, linewidth=1, c="r", label="NZDUSD")
plt.plot(1 / forex.USDCAD, linewidth=1, c="b", label="USDCAD") # 1/USDCAD=CADUSD
plt.xlabel("Time")
```

```
plt.ylabel("NZDUSD & AUDUSD & USDCAD Forex Rates")
plt.legend()
plt.show()
```



In figure 2, we can see all three time series seem to move together. Before running a cointegration test, we would like to check CADUSD to see if it is stationary. Let's run ADF on CADUSD.

Figure 3: Augmented Dickey Fuller Test for CADUSD and First Difference of CADUSD

```
In [4]: # Augmented Dickey Fuller Test for CADUSD and First Difference of CADUSD
CADUSD_adf = ADF(1 / forex.USDCAD, trend="n", method="bic")
print(
    "CADUSD Augmented Dickey-Fuller Unit Root Test\n", CADUSD_adf.regression.summary()
)
print("\nTest statistics and critical values: \n", CADUSD_adf)

print("\n", "# " * 39, "\n")
CADUSD_d_adf = ADF((1 / forex.USDCAD).diff().dropna(), trend="n", method="bic")
print(
    "First Difference of CADUSD Augmented Dickey-Fuller Unit Root Test\n",
    CADUSD_d_adf.regression.summary(),
)
print("\nTest statistics and critical values: \n", CADUSD_d_adf)
```

CADUSD Augmented Dickey-Fuller Unit Root Test

OLS Regression Results

=====						
===						
Dep. Variable:	y	R-squared (uncentered):	0.			
000						
Model:	OLS	Adj. R-squared (uncentered):	-0.			
001						
Method:	Least Squares	F-statistic:	0.0005			
211						
Date:	Tue, 23 Dec 2025	Prob (F-statistic):	0.			
982						
Time:	08:56:47	Log-Likelihood:	634			
0.4						
No. Observations:	1497	AIC:	-1.268e			
+04						
Df Residuals:	1496	BIC:	-1.267e			
+04						
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Level.L1	2.691e-06	0.000	0.023	0.982	-0.000	0.000
=====						
Omnibus:	142.495	Durbin-Watson:	1.980			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	978.432			
Skew:	0.036	Prob(JB):	3.44e-213			
Kurtosis:	6.960	Cond. No.	1.00			
=====						

Notes:

[1] R² is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:
Augmented Dickey-Fuller Results

=====	
Test Statistic	0.023
P-value	0.692
Lags	0

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

#####

First Difference of CADUSD Augmented Dickey-Fuller Unit Root Test
OLS Regression Results

=====	
===	

```

Dep. Variable:          y    R-squared (uncentered):          0.
495
Model:                  OLS    Adj. R-squared (uncentered):          0.
495
Method:                  Least Squares    F-statistic:          14
67.
Date:                    Tue, 23 Dec 2025    Prob (F-statistic):          3.17e-
224
Time:                    08:56:47    Log-Likelihood:          633
6.0
No. Observations:        1496    AIC:          -1.267e
+04
Df Residuals:            1495    BIC:          -1.266e
+04
Df Model:                1
Covariance Type:         nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Level.L1      -0.9905      0.026     -38.301      0.000     -1.041     -0.940
=====
Omnibus:                143.775    Durbin-Watson:          1.999
Prob(Omnibus):           0.000    Jarque-Bera (JB):          1002.086
Skew:                   0.028    Prob(JB):                2.51e-218
Kurtosis:               7.009    Cond. No.                 1.00
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:

Augmented Dickey-Fuller Results

```

=====
Test Statistic          -38.301
P-value                 0.000
Lags                    0
-----

```

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

From figure 3, we can see that the CADUSD level has a unit root whereas the CADUSD first difference is stationary. Hence, we can say CADUSD is $I(1)$. Now, we can proceed to conduct a Johansen test for cointegration. We will use a trace test in this example.

As we discussed in the last section, we need to decide the number of lags in the VEC model before we can use it to test cointegration. In the last lesson, we know R has the function to select the number of lags for a VAR model. We will use this function for our case.

Figure 4: Selection of Number of Lags for VEC Model

```
In [5]: # Selection of Number of Lags for VEC Model
vecm_data = pd.concat(
    [forex.AUDUSD, forex.NZDUSD, 1 / forex.USDCAD.rename("CADUSD")], axis=1
)

# Fit VAR model and run lag selection tool
model = VAR(vecm_data)
x = model.select_order(maxlags=7, trend="c")
x.summary()
```

```
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Value
Warning: A date index has been provided, but it has no associated frequency informat
ion and so will be ignored when e.g. forecasting.
self._init_dates(dates, freq)
```

```
Out[5]: VAR Order Selection (* highlights the
          minimums)

          AIC      BIC      FPE      HQIC
0  -23.49  -23.48  6.278e-11  -23.49
1  -34.83* -34.79*  7.474e-16* -34.81*
2  -34.83  -34.75  7.490e-16  -34.80
3  -34.83  -34.72  7.491e-16  -34.79
4  -34.82  -34.68  7.538e-16  -34.77
5  -34.81  -34.64  7.591e-16  -34.75
6  -34.81  -34.60  7.644e-16  -34.73
7  -34.80  -34.56  7.704e-16  -34.71
```

From figure 4, we can see all the information criteria select lag 1 for the level of the VAR model.

Now let's check Johansen trace test results.

Figure 5: Johansen Trace Test Result for AUDUSD, NZDUSD, and CADUSD

```
In [6]: # Johansen Trace Test Result for AUDUSD, NZDUSD, and CADUSD

jtest = coint_johansen(vecm_data, det_order=0, k_ar_diff=1)

# Print the results
print(f"Eigenvalues of VECM coefficient matrix : {jtest.eig}\n")

pd.DataFrame(
    {
        "Test statistic": jtest.trace_stat,
        "Critical values (90%)": jtest.trace_stat_crit_vals[:, 0],
    })
```



```

        "Critical values (95%)": jtest.trace_stat_crit_vals[:, 1],
        "Critical values (99%)": jtest.trace_stat_crit_vals[:, 2],
    },
    index=["rank=0", "rank<=1", "rank<=2"],
)

```

Eigenvalues of VECM coefficient matrix : [0.01205939 0.00675233 0.00307245]

Out[6]:

	Test statistic	Critical values (90%)	Critical values (95%)	Critical values (99%)
rank=0	32.889720	27.0669	29.7961	35.4628
rank<=1	14.739214	13.4294	15.4943	19.9349
rank<=2	4.603465	2.7055	3.8415	6.6349

In [7]:

```

# VECM model
vecm_model = VECM(endog=vecm_data, k_ar_diff=1, deterministic="ci").fit()
print(vecm_model.summary())

```

/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Value Warning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.
self._init_dates(dates, freq)

Det. terms outside the coint. relation & lagged endog. parameters for equation AUDUSD

	coef	std err	z	P> z	[0.025	0.975]
L1.AUDUSD	0.0084	0.048	0.175	0.861	-0.086	0.103
L1.NZDUSD	-0.0666	0.045	-1.477	0.140	-0.155	0.022
L1.CADUSD	0.0878	0.040	2.189	0.029	0.009	0.166

Det. terms outside the coint. relation & lagged endog. parameters for equation NZDUSD

	coef	std err	z	P> z	[0.025	0.975]
L1.AUDUSD	0.0317	0.047	0.674	0.501	-0.061	0.124
L1.NZDUSD	-0.0647	0.044	-1.464	0.143	-0.151	0.022
L1.CADUSD	0.0779	0.039	1.985	0.047	0.001	0.155

Det. terms outside the coint. relation & lagged endog. parameters for equation CADUSD

	coef	std err	z	P> z	[0.025	0.975]
L1.AUDUSD	0.0405	0.040	1.009	0.313	-0.038	0.119
L1.NZDUSD	-0.0203	0.038	-0.540	0.589	-0.094	0.053
L1.CADUSD	-0.0088	0.033	-0.263	0.793	-0.074	0.057

Loading coefficients (alpha) for equation AUDUSD

	coef	std err	z	P> z	[0.025	0.975]
ec1	0.0021	0.008	0.270	0.787	-0.013	0.017

Loading coefficients (alpha) for equation NZDUSD

	coef	std err	z	P> z	[0.025	0.975]
ec1	0.0198	0.008	2.601	0.009	0.005	0.035

Loading coefficients (alpha) for equation CADUSD

	coef	std err	z	P> z	[0.025	0.975]
ec1	0.0089	0.006	1.367	0.172	-0.004	0.022

Cointegration relations for loading-coefficients-column 1

	coef	std err	z	P> z	[0.025	0.975]
beta.1	1.0000	0	0	0.000	1.000	1.000
beta.2	-1.1497	0.131	-8.779	0.000	-1.406	-0.893
beta.3	-0.1245	0.180	-0.690	0.490	-0.478	0.229
const	0.1539	0.104	1.487	0.137	-0.049	0.357

Figure 5 shows the result of the Johansen trace test. The top part in Figure 5 shows the result for three tests: H_0 : rank = 0, H_0 : rank = 1, and H_0 : rank = 2. Let's use 5% as our decision point. For H_0 : rank = 0 we can see the test statistic is 32.8897 and the 5% critical value is 29.7961. We reject H_0 and say the rank is not 0 or a cointegration relationship would exist. Next, we investigate H_0 : rank = 1. The test statistic is 14.7392

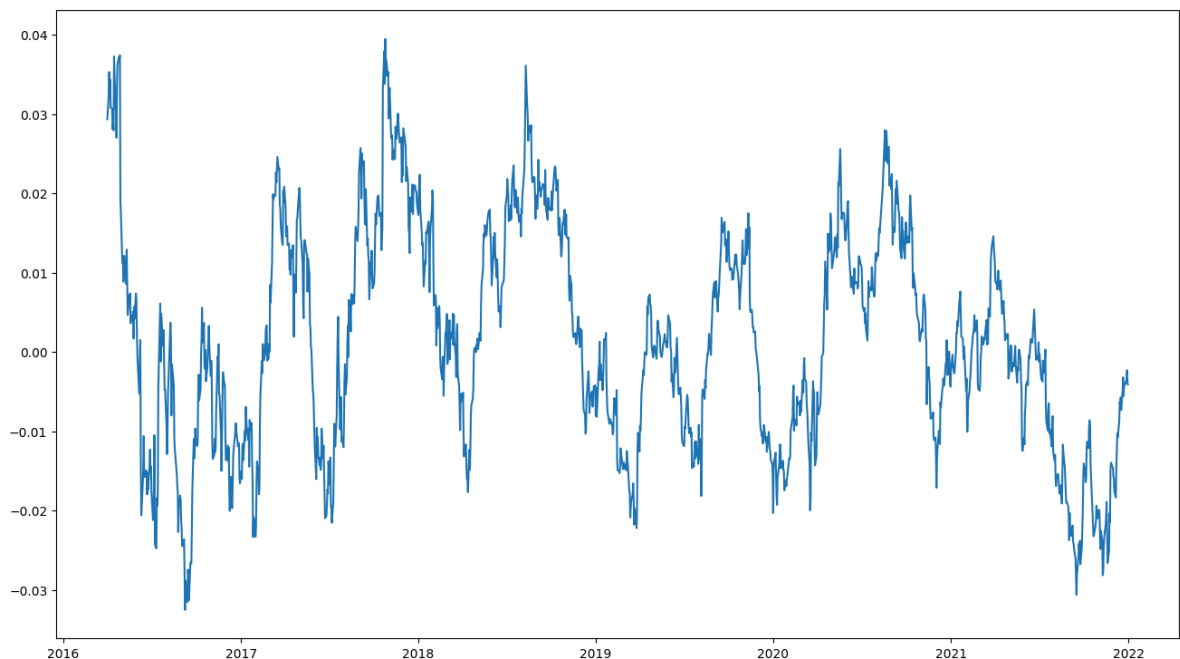
and the 5% critical value is 15.4943. Hence, we cannot reject H_0 : rank = 1. There is one linear combination of three time series variables that is stationary. The coefficients of the linear combination are in the second part under '*Cointegration relations for loading-coefficients-column 1*' heading. We can write the linear combination as follows:

$$S = 0.1539131 + 1 \cdot \text{AUDUSD} - 1.1497011 \cdot \text{NZDUSD} - 0.1245218 \cdot \text{CADUSD}$$

The above equation is the deviation from the long-term equilibrium of the three time series. Let's check out the plot to see if this deviation is stationary.

Figure 6: Time Plot for Deviation from Long-Term Equilibrium

```
In [8]: # Time Plot for Deviation from Long-Term Equilibrium
        """
        The precise coefficients of the linear combination are in:
        vecm_model.const_coint and vecm_model.beta
        S = 0.1539131 + 1*vecm_data.AUDUSD - 1.1497011*vecm_data.NZDUSD - 0.1245218*vecm_data.CADUSD
        """
        S = (
            vecm_model.const_coint[0][0]
            + vecm_model.beta[0][0] * vecm_data.AUDUSD
            + vecm_model.beta[1][0] * vecm_data.NZDUSD
            + vecm_model.beta[2][0] * vecm_data.CADUSD
        )
        plt.plot(S)
        plt.show()
```



From figure 6, we can see the deviation from long-term equilibrium is moving within a channel. Let's take a look at the ADF test for the deviation.

Figure 7: ADF Test Result for Deviation from Long-Term Equilibrium

```
In [9]: # ADF Test Result for Deviation from Long-Term Equilibrium
S_adf = ADF(S, trend="n", method="bic")
print("Augmented Dickey-Fuller Unit Root Test\n", S_adf.regression.summary())
print("\nTest statistics and critical values: \n", S_adf)
```

Augmented Dickey-Fuller Unit Root Test

OLS Regression Results

```
=====
===
Dep. Variable:          y      R-squared (uncentered):          0.
012
Model:                  OLS    Adj. R-squared (uncentered):      0.
011
Method:                 Least Squares    F-statistic:          1
7.62
Date:                   Tue, 23 Dec 2025    Prob (F-statistic):      2.85e
-05
Time:                   08:56:48    Log-Likelihood:          667
1.8
No. Observations:       1497    AIC:                      -1.334e
+04
Df Residuals:           1496    BIC:                      -1.334e
+04
Df Model:                1
Covariance Type:        nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Level.L1      -0.0217      0.005     -4.198      0.000     -0.032     -0.012
=====
Omnibus:                128.993    Durbin-Watson:           2.019
Prob(Omnibus):           0.000    Jarque-Bera (JB):         759.418
Skew:                   -0.087    Prob(JB):                 1.24e-165
Kurtosis:                6.485    Cond. No.                  1.00
=====
```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test statistics and critical values:

Augmented Dickey-Fuller Results

```
=====
Test Statistic          -4.198
P-value                  0.000
Lags                     0
-----
```

Trend: No Trend

Critical Values: -2.57 (1%), -1.94 (5%), -1.62 (10%)

Null Hypothesis: The process contains a unit root.

Alternative Hypothesis: The process is weakly stationary.

From figure 7, we can see that we easily reject that the deviation has a unit root because the test statistic (-4.198) is below 5% critical value (-1.94).

5. Shiny Application for Unit Root Test and Johansen Cointegration Test

In this module, we provided a Shiny application to run a unit root test and Johansen cointegration test.

[Click here to access the application.](#) For the unit root test tab, you select a variable from the variable list. The application will show augmented Dickey Fuller test results for both the level and first difference of the variable. For the Johansen cointegration test tab, you select two variables to run the test, and then the application will show the result.

6. Conclusion

In this lesson, we expanded our cointegration analysis to multiple time series. We first talked about the meaning of cointegration in the case of multiple time series. We then introduced the vector error correction model (VECM) as a foundation to understand the analysis of cointegration in multiple time series. We discussed the rank of Π coefficient matrix and the meanings of different ranks. We then introduced Johansen cointegration tests. We talked about two types of Johansen cointegration tests: trace test and maximum eigenvalue test. We showed how both tests are set up. We finished the lesson with one application of the Johansen trace test and demonstrated how to read the test results. This lesson concludes module 6. In the next module, we will introduce another model: agent-based modeling.

References

- Johansen, Søren. "A Statistical Analysis of Integration for I(2) Variables." *Econometric Theory*, vol. 11, no. 1, 1995, pp. 25–59.